



北京大学
PEKING UNIVERSITY

《AI化学实践》

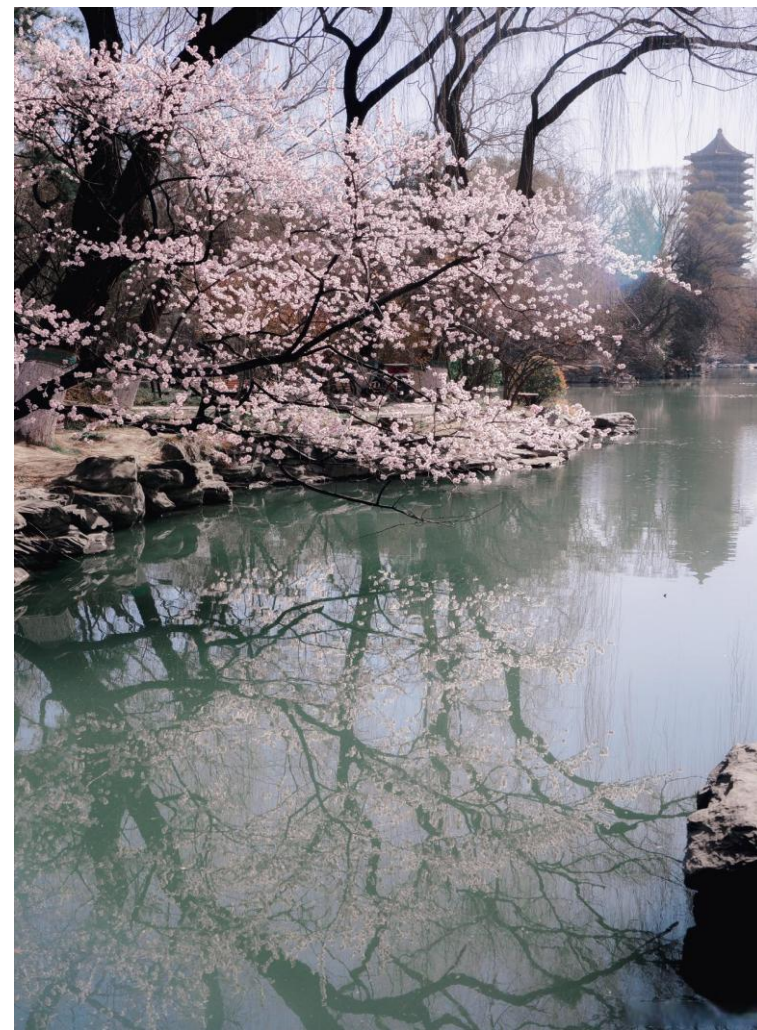
细胞图像智能分析实验项目

一、深度学习图像分析基础

2026.08.24-08.28

大纲

- 项目介绍
- 实验一
- 实验二
- 实验三



课前问卷



课程简介

《AI化学实践》

- 首次开课：2025暑期
- 学分：1
- 学时：32（实验课）
- 成绩评定：百分制
- 实验项目：五选一
- 项目五：细胞图像智能分析
- 项目五教师：李霄、彭陆鑫（邹鹏老师组）

项目负责人：李霄



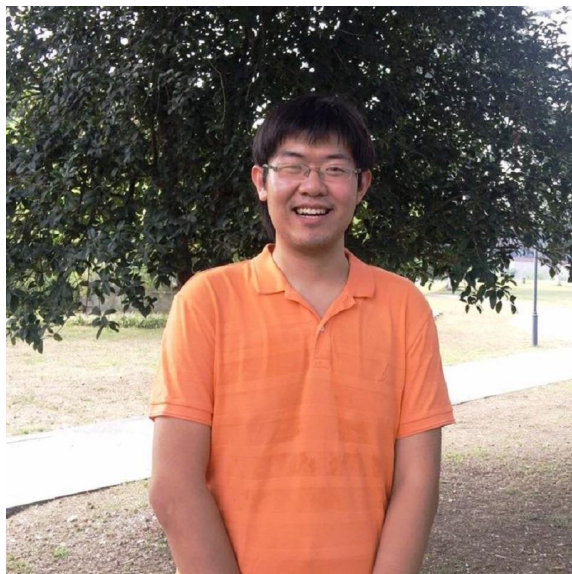
- 2007-2011, 学士, 香港城市大学, 生物与化学系
- 2011-2016, 博士, 美国加州大学默赛德分校, 化学与化学生物学系
- 2018-2021, 博士后, 美国罗格斯大学, 化学与化学生物学系
- 2022-2023, 讲师, 西湖大学, 理学院化学系
- 2024 -至今, 讲师, 北京大学, 化学与分子工程学院

研究领域：物理有机化学，化学教育研究

邮箱：xiaoli@pku.edu.cn

办公室：北京大学化学与分子工程学院 B415

指导教师：彭陆鑫



- 2010-2014, 学士, 南京大学, 化学化工学院
- 2016-2023, 博士, 北京大学, 化学与分子工程学院
- 2023-至今, 博士后, 北京大学, 化学与分子工程学院 (邹鹏老师组)

研究领域：化学生物学

邮箱：pengluxin@pku.edu.cn

办公室：C510

项目安排

- 上课地点: B300(第一天) , B600 (其他天)
- 时间安排

天数	日期	时间	内容
Day1	8月24日	10:10-17:00	实践课程
Day2	8月25日	10:10-17:00	实践课程
Day3	8月26日	10:10-17:00	实践课程
Day4	8月27日	10:10-13:00	实践课程
		14:00-18:00	自主讨论实践
Day5	8月28日	9:00-14:00	自主讨论实践
		14:00-18:00	汇报讨论
-	8月30日	18:00前	提交书面报告

课程设置与安排

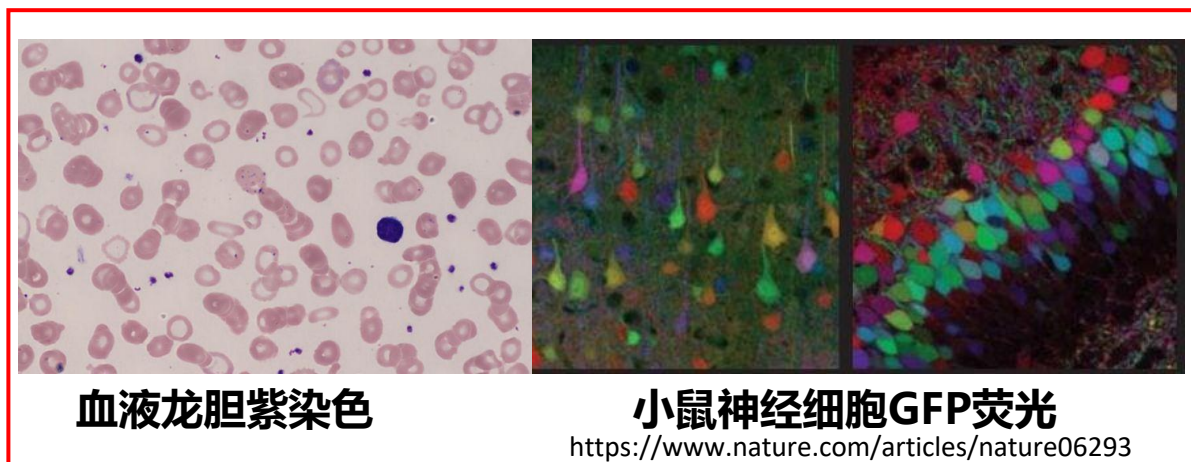
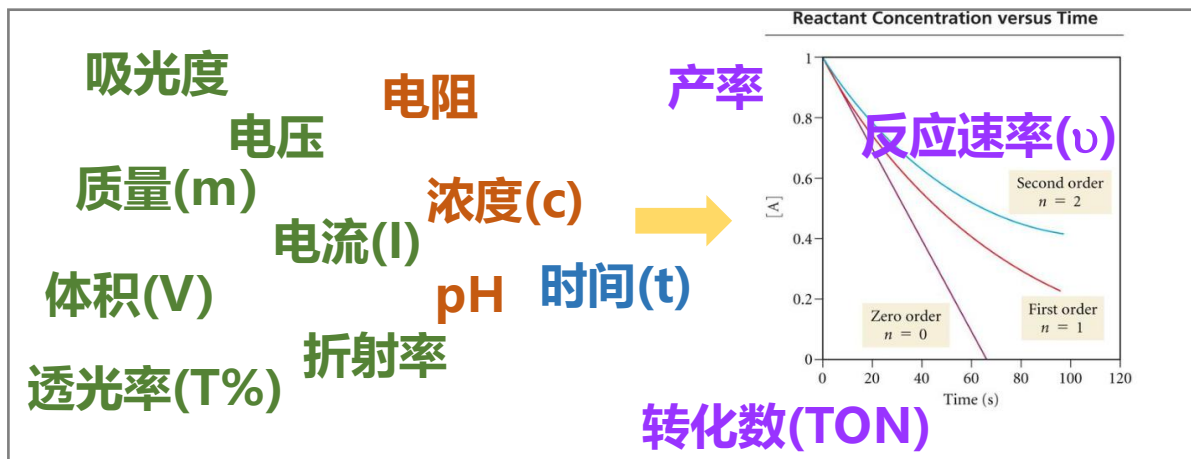
- **模块一：人工智能图像分析的理论 and 工程基础 (Day 1-2, 李霄)**
 - 实验一 Lab1: 机器学习在图像分类中的应用
 - 实验二 Lab2: 深度学习在图像分析中的应用
 - 实验三 Lab3: 细胞分割大模型在细胞图像分析中的应用
- **模块二：图像分析在化学生物学实验中的应用 (Day 3-4, 陆彭鑫)**
 - 实验四 Lab4: 应用图像分割算法 (Cellpose) 助力荧光膜电位探针的高通量筛选
- **总结汇报**
 - 口头汇报 (Day 5): 小组
 - 书面报告(7.6): 个人

课程及项目要求

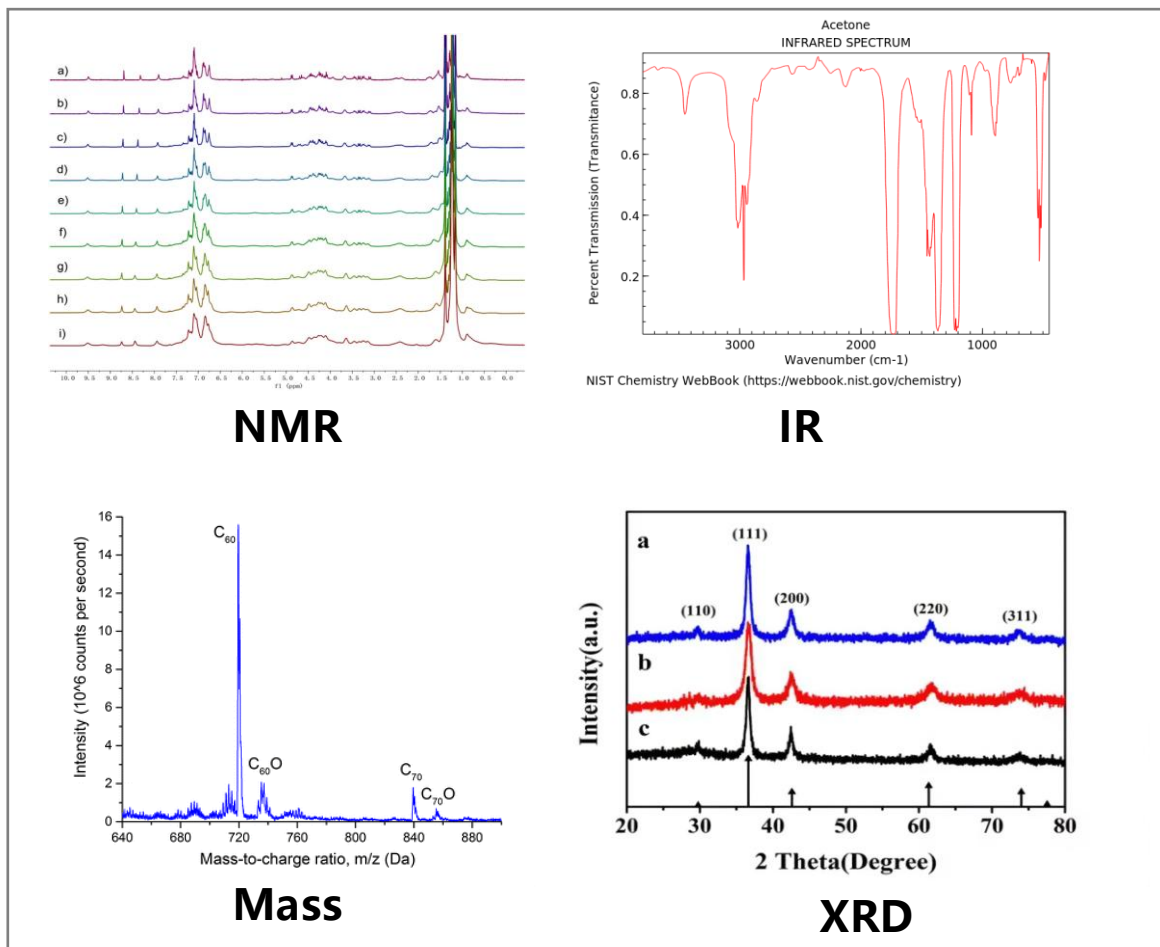
评分标准

- **实践过程 65%**
 - 提交实验笔记 (以.ipynb格式)
 - Lab 1(by 8.24), lab 2&3(by 8.25),
- **书面报告 (每人单独提交) 15%**
 - 模块一, 图像分析的深度学习算法程序运行的结果和分析
 - 模块二, 深度学习模型运用于分析真实实验数据的方法、结果和分析
- **口头汇报 (小组完成) 20%**
 - 成员合作完成幻灯片的制作, 1~2名同学代表项目成员进行口头汇报, 约30分钟

背景：化学实验的数据处理

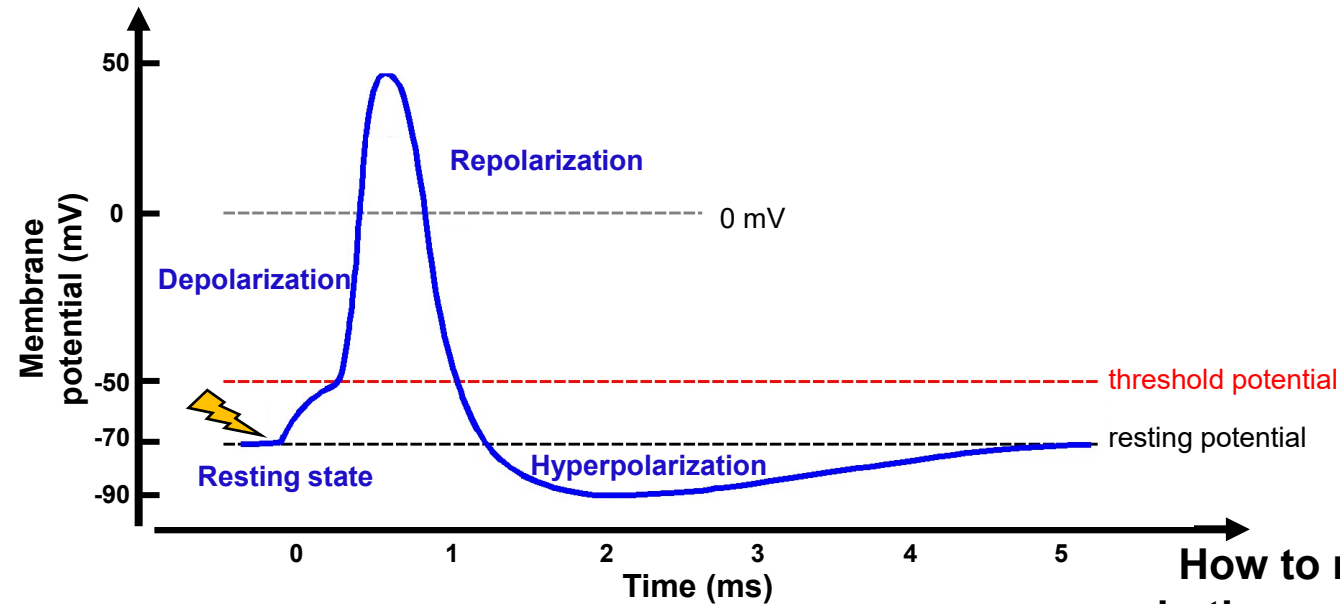


显微图像



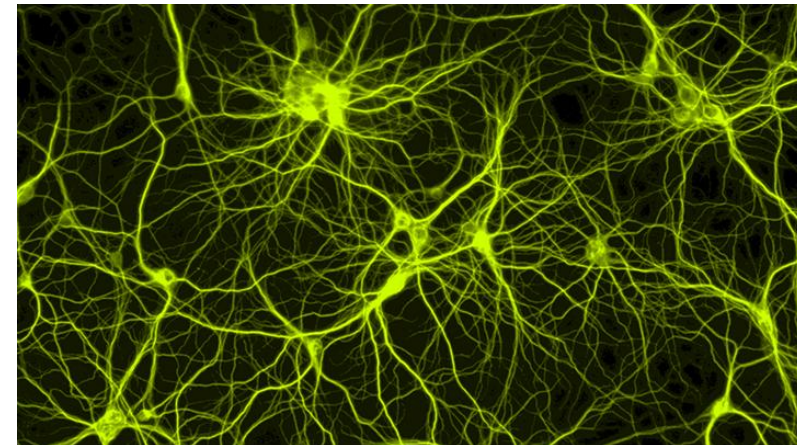
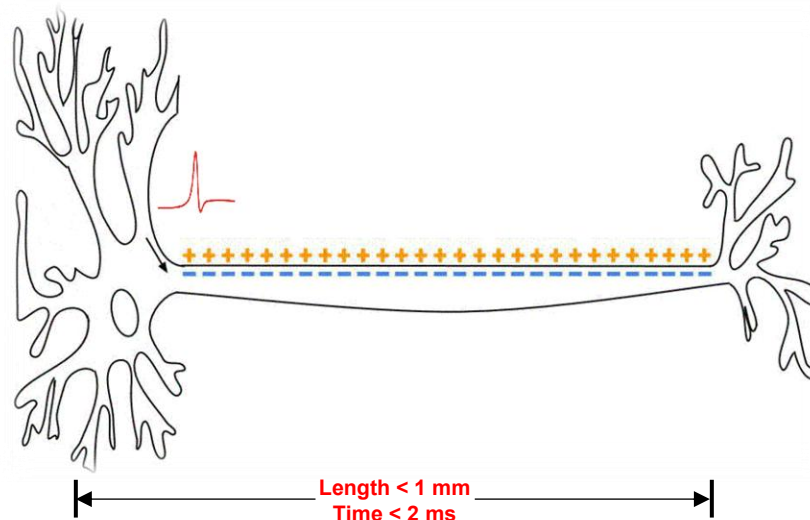
谱图

Neurons Communicate via Action Potentials

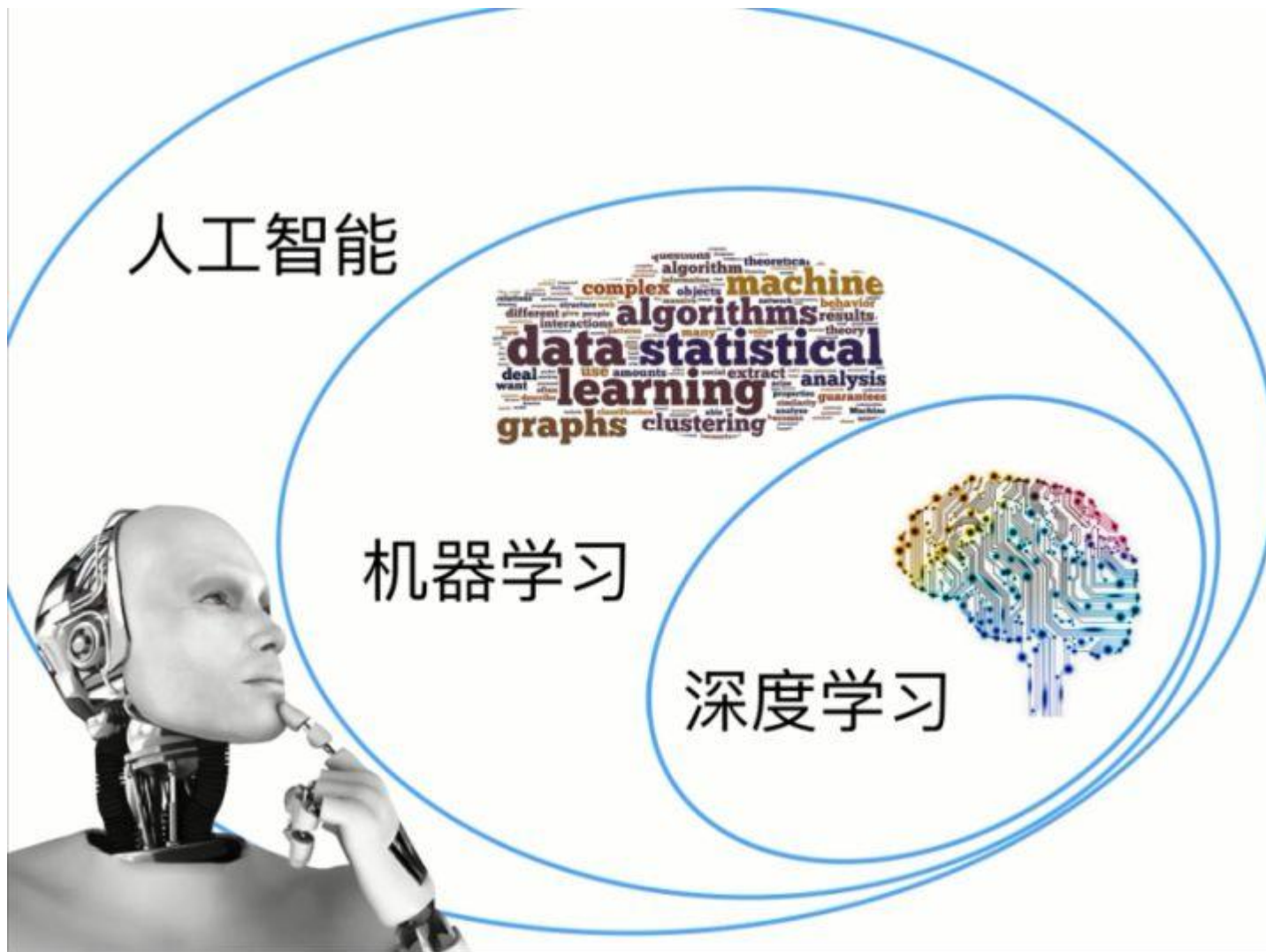


陆彭鑫 (邹鹏课题组)

How to record action potentials
in the complex neuronal network?

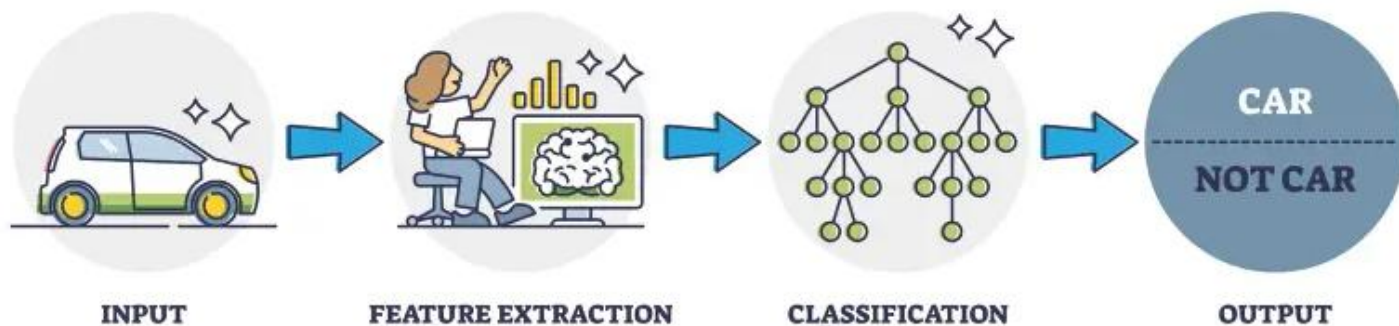


人工智能



人工智能在图像分析中的应用

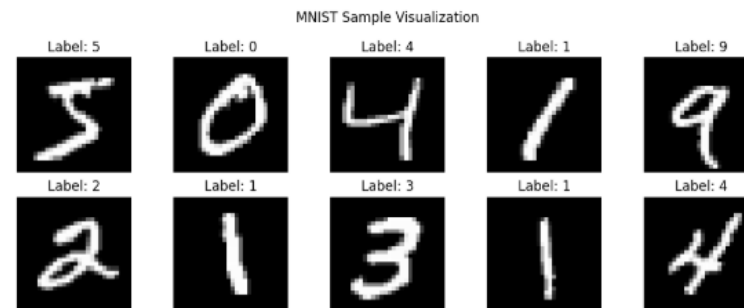
MACHINE LEARNING



DEEP LEARNING



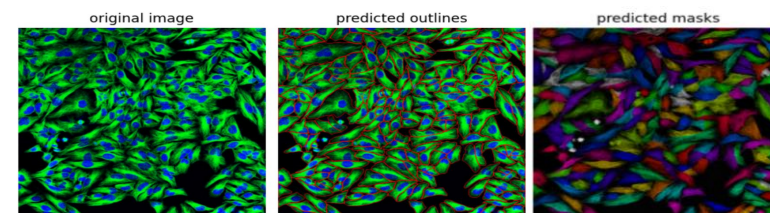
手写数字: load_digits, MNIST



彩色图片: CIFAR-10



细胞图片: cellpose数据集, 北大化院实验数据

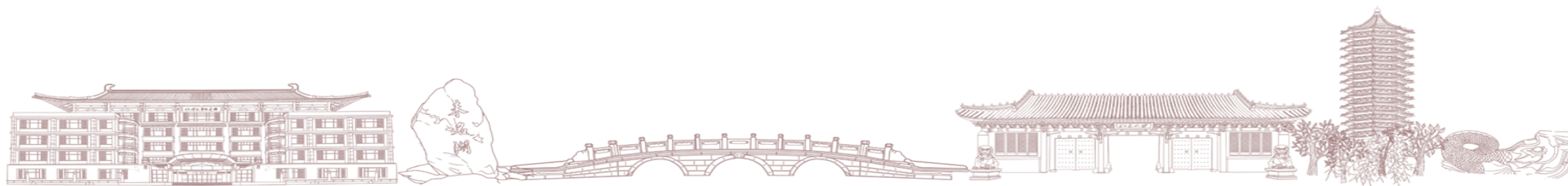




实验一、机器学习在图像分类中的应用

Lab 1: Machine Learning for Image Classification

Jupyter notebook: lab1_ML4ImgRec.ipynb



上课方式

深度学习在化学生物学细胞图像分析中的应用

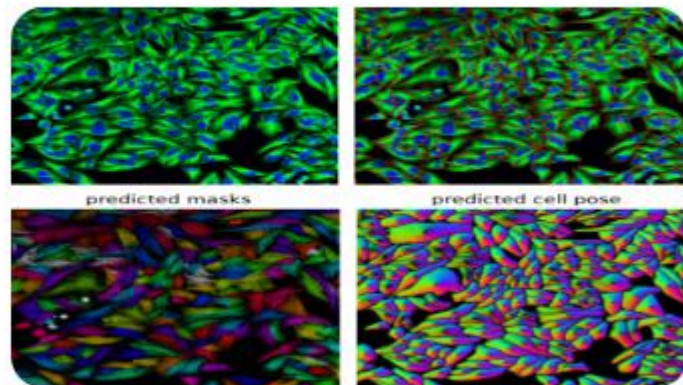
通过介绍机器学习，多层感知机(MLP)，卷积神经网络(CNN)在图像分析中的原理，进一步介绍细胞荧光图像的分类和分割应用。

授课教师  李霄、 赵泽华、彭陆鑫

开课时间 2025-06-28 已报名 8

进入课程

管理课程



深度学习在化学生物学细胞图像分析中的...

Lab01: Machine Learning for Image Cl...

lab1_ML4ImgRec
Notebook

Lab02: Introduction to Deep Learning ...

lab2_intro2DL-cpu
Notebook

lab2_intro2DL-gpu
Notebook

Lab03: Introduction to Cell Segmentat...

lab3_intro2CellSeg
Notebook

公开 lab1_ML4ImgRec

python Machine Learning Image Classification

xiao_li

更新于2025-06-28

推荐镜像: Basic Image:bohrium...

推荐机型: c2_m4_cpu

≡

⏪ ⏴ ⏵ ⏩

开始连接

AI for Cell Image Analysis

Dr. Xiao Li xiaoli@pku.edu.cn, 2025 Summer

Lab Notebook 01

方式一:

- 登录波尔Bohrium平台
- 在 课程 中搜索 “深度学习在化学生物学细胞图像分析中的应用”
- 课程链接:

<https://www.bohrium.com/courses/9845621078/content/update?tab=courseinfo&classId=9845621078>

上课方式

方式二：

把课程资料下载到本地，资料文件夹里包含jupyter notebooks和./data, ./img文件夹。使用VS Code或者Jupyter Notebook软件打开notebook。你需要安装3.1以上的版本的python以及实验所需函数包。

先修课程资源

科学编程与化学机器学习入门

一门精炼课程，介绍Python科学编程与机器学习基础，通过基于化学的编程实践练习帮助学习者上手Python和主要使用scikit-learn函数库的机器学习。适用于：化学背景对科学编程、机器学习感兴趣的学者。预计学习时长：小于8小时。

授课教师 李霄

课程状态 开课中 开课时间 2024-06-30 已报名 182

预览

进入课程



课程链接：

<https://www.bohrium.com/courses/3808834495/content/update?tab=courseware&classId=3808834495>

Jupyter Notebook 交互式笔记本

.ipynb

- Creating Lists

A list is created by placing elements inside square brackets. Below, the list called `mass` is created containing the atomic mass of the first six chemical elements. Type the list name `mass` in the code cell below to print out all the element in the list.

标记文本单元格

+ Code | + Markdown

```
[ ] 1 mass = [1.01, 4.00, 6.94, 9.01, 10.81, 12.01] #note square brackets
    2 # type the list name "mass" below to print out all the elements:
    3 mass
```

代码单元格 (Python)

Jupyter Notebook 交互式笔记本

- **练习：**添加一个文本单元格，写上你的姓名和学号
- 双击文本单元格进行编辑
- 保存notebook副本

```
<a id=0> </a>
**AI for Cell Image Analysis** <br>
Dr. Xiao Li xiaoli@pku.edu.cn, 2025 Summer

<span style="font-size:20px">Lab Notebook 01</span>

_Student name:_<font color = 'blue'> (input your name here)</font> <br>
_Student ID:_<font color = 'blue'> (input your student ID here)</font> <br>
```

标记语言

AI for Cell Image Analysis
Dr. Xiao Li xiaoli@pku.edu.cn, 2025 Summer

Lab Notebook 01

Student name: (input your name here)
Student ID: (input your student ID here)

显示效果

笔记本中标记<To-Do> 字样的部分是需要同学同步完成的笔记任务

导入函数库

```
##You don't need to run this cell if you run this notebook on Bohrium or you have previously installed these libraries
```

```
#pip install numpy  
#pip install matplotlib  
#pip install pandas  
#pip install seaborn  
#pip install sklearn
```

```
import numpy as np  
import matplotlib.pyplot as plt  
import pandas as pd  
import seaborn as sns  
from sklearn.model_selection import train_test_split  
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
```

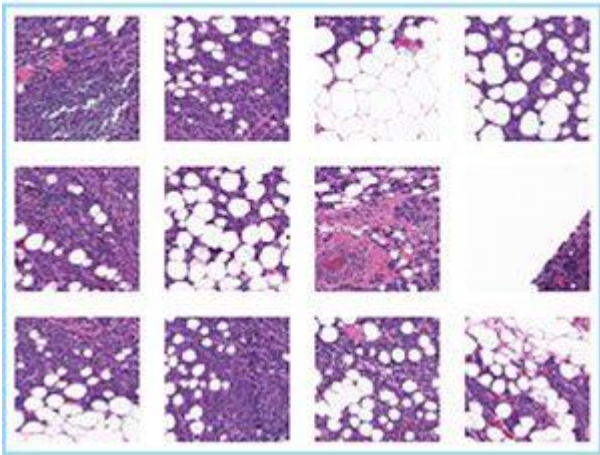
二元分类问题：逻辑回归 Logistic Regression

特征名称	中文解释	特征含义描述
mean radius	平均半径	肿块细胞核的平均半径（单位：像素，所有测量点的平均值）。
mean texture	平均纹理	灰度值的标准差，反映细胞核图像的纹理复杂度（灰度变化的剧烈程度）。
mean perimeter	平均周长	细胞核边界的平均周长（单位：像素），与半径呈正相关，但更能反映轮廓的复杂程度。
mean area	平均面积	细胞核的平均面积（单位：平方像素），通常与肿瘤大小相关。
mean smoothness	平均平滑度	半径变化的平滑程度（即轮廓的起伏程度），数值越小表示轮廓越规则。
mean compactness	平均紧凑度	基于周长和面积的计算值（公式： $4\pi \times \text{面积} / \text{周长}^2 - 1$ ），用于衡量形状的紧凑性，值越大表示形状越不规则。
mean concavity	平均凹度	轮廓凹陷部分的严重程度（如凹痕的深度），反映细胞核形态的异常程度。
mean concave points	平均凹点数量	轮廓上凹点的数量（如凹陷的个数），恶性肿瘤通常有更多不规则凹点。
mean symmetry	平均对称性	细胞核形状的对称性，良性肿瘤通常更对称，恶性肿瘤对称性较差。

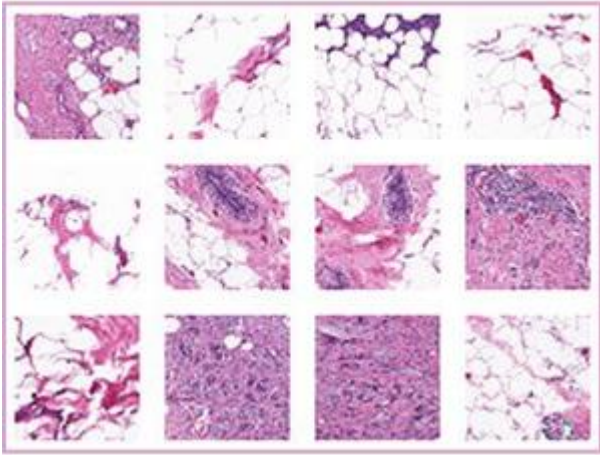
.....

乳腺组织纤维图像

恶性 (malignant)



良性 (Benign)



Sklearn load_breast_cancer 数据集

```
cancer = load_breast_cancer()
```

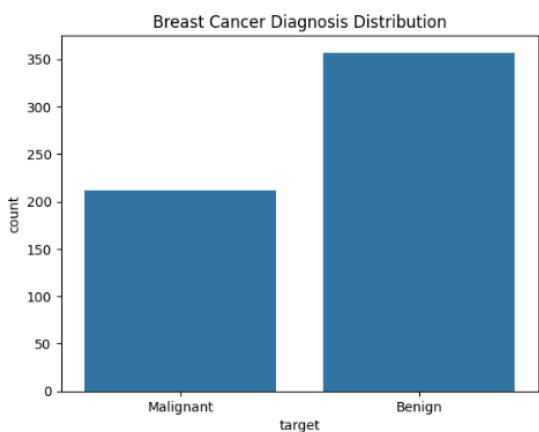
569个样品， 30个特征， 1列目标（0恶性， 1良性）

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst texture	worst perimeter	worst area	worst smoothness	worst compactness	worst concavity	worst concave points	worst symmetry	worst fractal dimension	target
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.30010	0.14710	0.2419	0.07871	...	17.33	184.60	2019.0	0.16220	0.66560	0.7119	0.2654	0.4601	0.11890	0
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.08690	0.07017	0.1812	0.05667	...	23.41	158.80	1956.0	0.12380	0.18660	0.2416	0.1860	0.2750	0.08902	0
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.19740	0.12790	0.2069	0.05999	...	25.53	152.50	1709.0	0.14440	0.42450	0.4504	0.2430	0.3613	0.08758	0
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.24140	0.10520	0.2597	0.09744	...	26.50	98.87	567.7	0.20980	0.86630	0.6869	0.2575	0.6638	0.17300	0
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.19800	0.10430	0.1809	0.05883	...	16.67	152.20	1575.0	0.13740	0.20500	0.4000	0.1625	0.2364	0.07678	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
564	21.56	22.39	142.00	1479.0	0.11100	0.11590	0.24390	0.13890	0.1726	0.05623	...	26.40	166.10	2027.0	0.14100	0.21130	0.4107	0.2216	0.2060	0.07115	0
565	20.13	28.25	131.20	1261.0	0.09780	0.10340	0.14400	0.09791	0.1752	0.05533	...	38.25	155.00	1731.0	0.11660	0.19220	0.3215	0.1628	0.2572	0.06637	0
566	16.60	28.08	108.30	858.1	0.08455	0.10230	0.09251	0.05302	0.1590	0.05648	...	34.12	126.70	1124.0	0.11390	0.30940	0.3403	0.1418	0.2218	0.07820	0
567	20.60	29.33	140.10	1265.0	0.11780	0.27700	0.35140	0.15200	0.2397	0.07016	...	39.42	184.60	1821.0	0.16500	0.86810	0.9387	0.2650	0.4087	0.12400	0
568	7.76	24.54	47.92	181.0	0.05263	0.04362	0.00000	0.00000	0.1587	0.05884	...	30.37	59.16	268.6	0.08996	0.06444	0.0000	0.0000	0.2871	0.07039	1

569 rows x 31 columns

X

y



Python dictionary 字典

```
# intro to dictionary
chemdict = dict(
    element_symbol=["H", "He", "Li", "Be", "B"],
    atomic_number=[1, 2, 3, 4, 5],
    element_ENname=["hydrogen", "helium", "lithium", "beryllium", "boron"],
    element_CNname=["氢", "氦", "锂", "铍", "硼"])
```


key键


value值

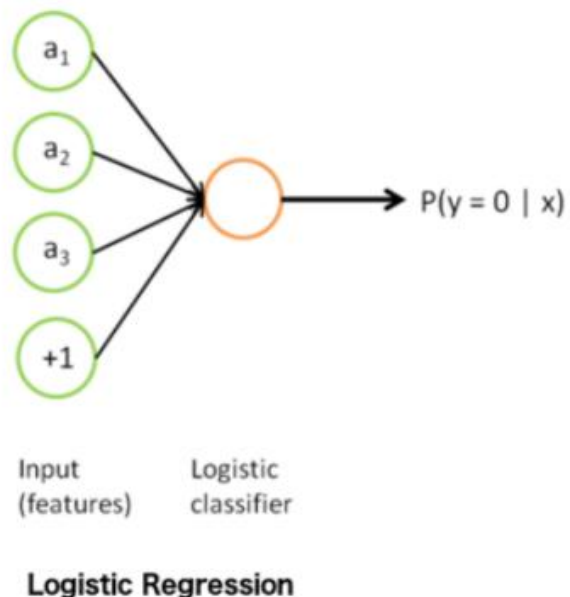
```
len(chemdict)
```

4 #字典共含4个键值对

```
chemdict["element_symbol"][0]
```

'H'

数据处理



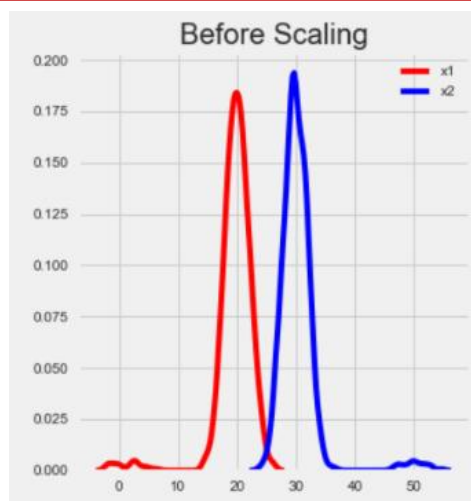
定义特征(X)和目标(y)

拆分训练集、测试集

```
cancer_X = cancer_df.drop('target', axis=1) #.drop cuts out the target column  
cancer_y = cancer_df['target']
```

```
#test_size=0.2 means 20% of the data is split into the test set  
cancer_X_train, cancer_X_test, cancer_y_train, cancer_y_test =  
train_test_split(cancer_X, cancer_y, test_size=0.2, random_state=42)
```

参数缩放 Parameter Scaling

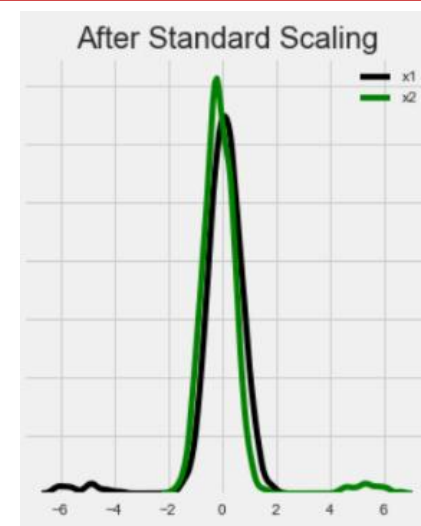


$$z = \frac{x - \mu}{\sigma}$$

μ = Mean

σ = Standard Deviation

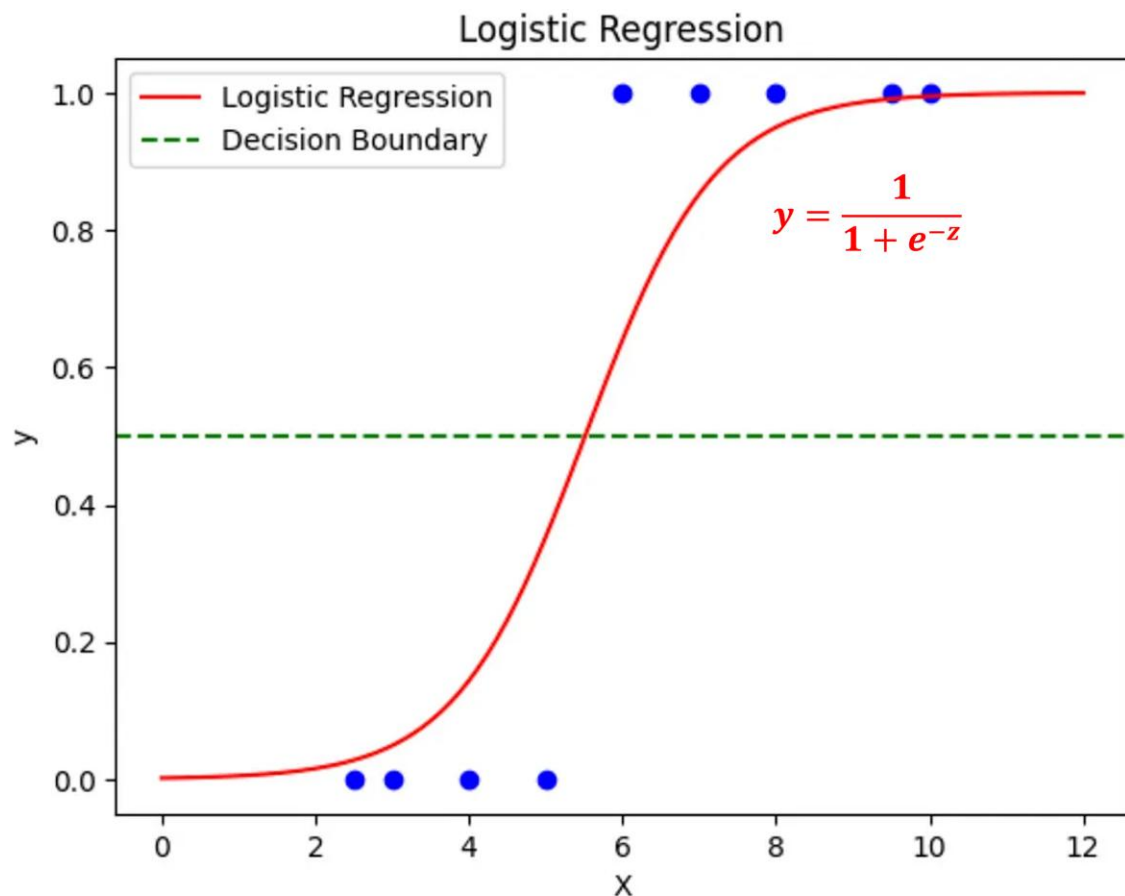
StandardScaler()



mean radius	mean texture	mean perimeter	mean area	mean smoothness \
12.47	18.60	81.09	481.9	0.09965
18.94	21.31	123.60	1130.0	0.09009
15.46	19.48	101.70	748.9	0.10920
12.40	17.68	81.47	467.8	0.10540
11.54	14.44	74.65	402.9	0.09984
...	...	...	...	...
14.64	16.85	94.21	666.0	0.08641
16.07	19.65	104.10	817.7	0.09168
11.52	14.93	73.87	406.3	0.10130
14.22	27.85	92.55	623.9	0.08223
20.73	31.12	135.70	1419.0	0.09469

```
[[-0.46649743 -0.13728933 -0.44421138 ... -0.19435087 0.17275669  
 0.20372995]  
[ 1.36536344 0.49866473 1.30551088 ... 0.99177862 -0.561211  
-1.00838949]  
[ 0.38006578 0.06921974 0.40410139 ... 0.57035018 -0.10783139  
-0.20629287]  
...  
[-0.73547237 -0.99852603 -0.74138839 ... -0.27741059 -0.3820785  
-0.32408328]  
[ 0.02898271 2.0334026 0.0274851 ... -0.49027026 -1.60905688  
-0.33137507]  
[ 1.87216885 2.80077153 1.80354992 ... 0.7925579 -0.05868885  
-0.09467243]]
```

逻辑回归 Sigmoid 函数



$$z = w_0x_0 + w_1x_1 + \dots + w_nx_n + b$$

Sigmoid 函数: $y = \frac{1}{1+e^{-z}}$

优化一组 $w_0, w_1, \dots, w_n$ 和 b , 是的
 y_{predict} 最接近 y_{true} .

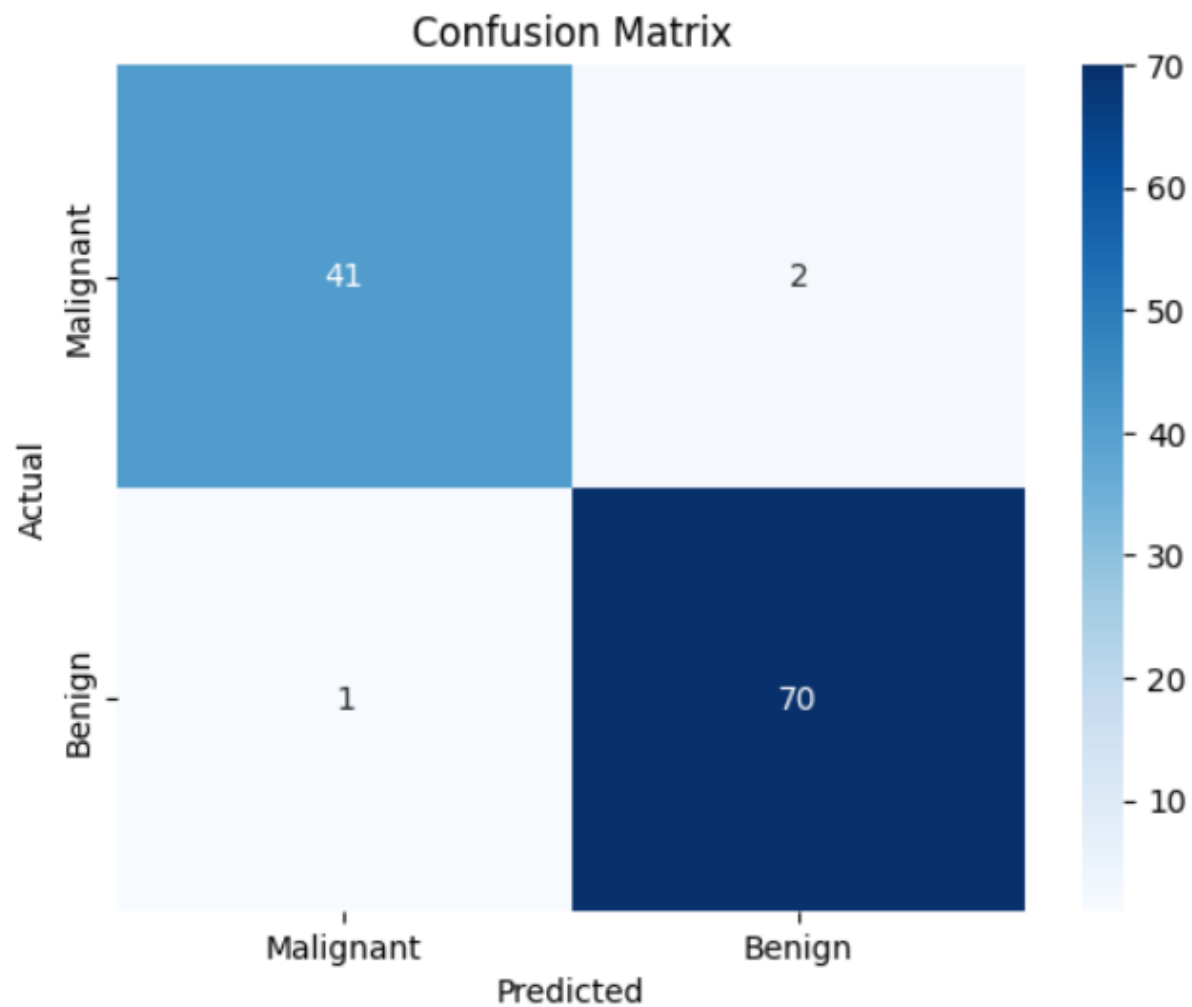
- $y > 0.5, y_{\text{predict}} = 1$
- $y \leq 0.5, y_{\text{predict}} = 0$

调用sklearn函数包

```
cancer_model = LogisticRegression()  
cancer_model.fit(cancer_X_train_scaled, cancer_y_train)  
  
cancer_y_pred = cancer_model.predict(cancer_X_test_scaled)  
print("Accuracy Score:", accuracy_score(cancer_y_test, cancer_y_pred))
```

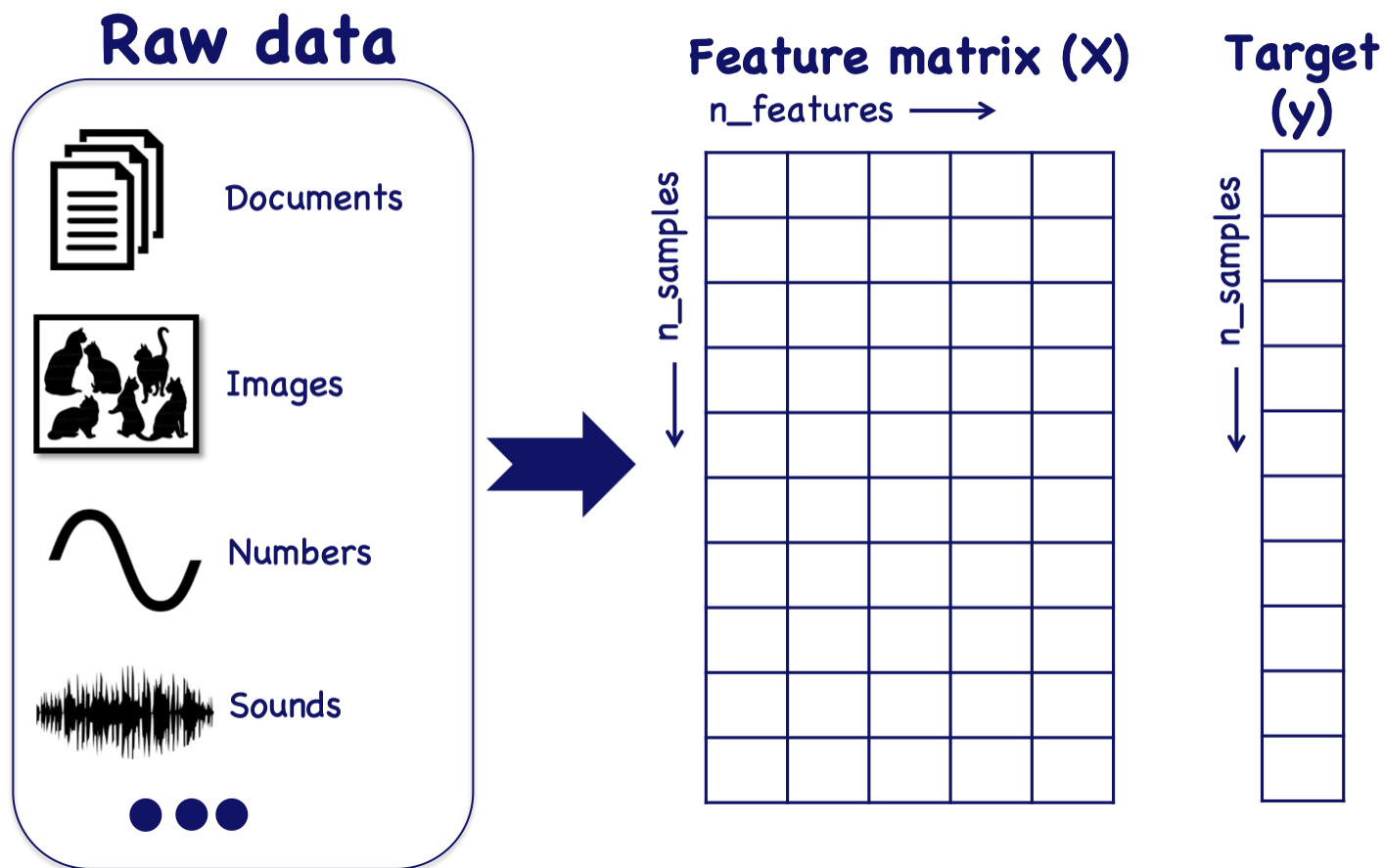
Accuracy Score: 0.9736842105263158

分类模型评估：混淆矩阵



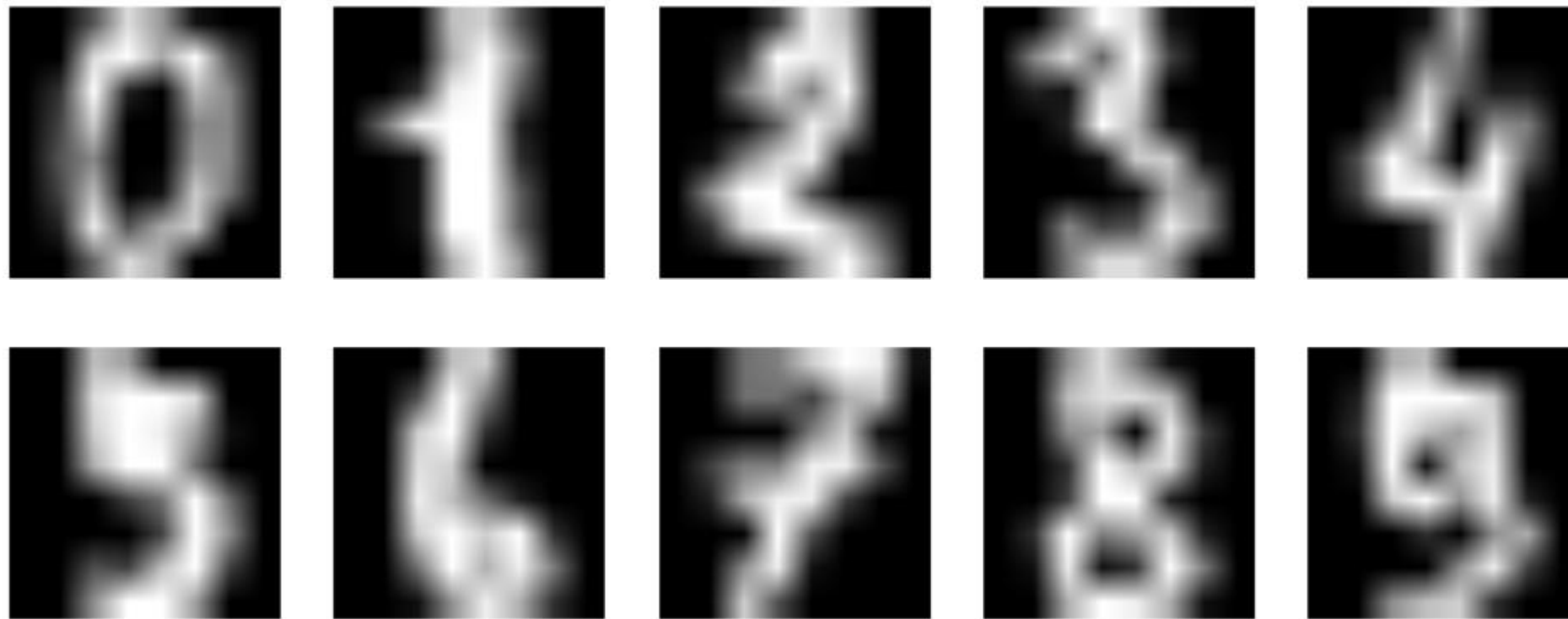
机器学习小节

数据处理（特征、目标、正则化） → 数据集、测试集 → 模型训练 → 预测、评估



多类别分类问题

Sample Digit Images



类别：共10类

Scikit-Learn load_digits 数据集

```
for key in digits:  
    print(key)
```

```
data  
target  
frame  
feature_names  
target_names  
images  
DESCR
```

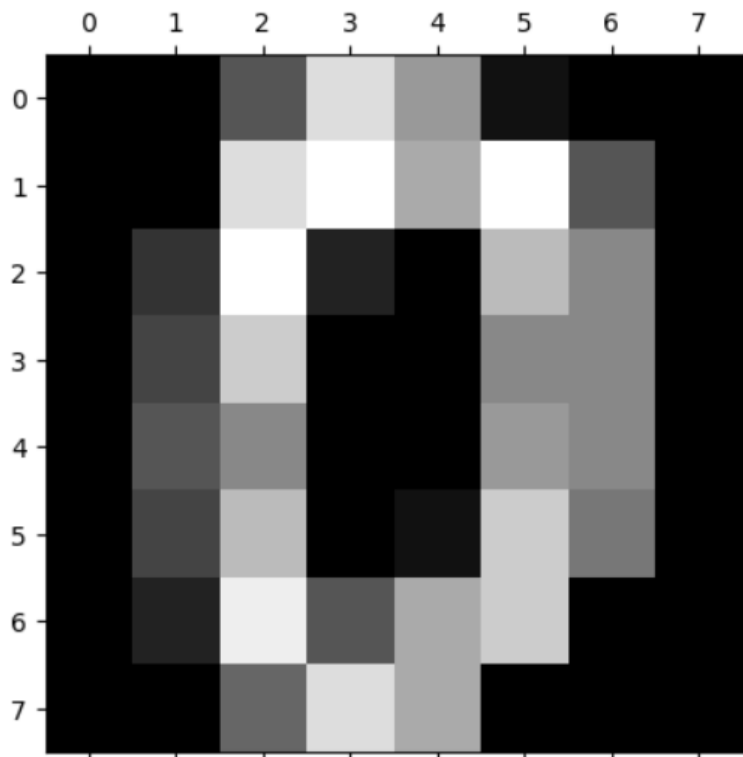
```
print(f"data shape of images: {digits.images.shape}")  
print(f"data shape of each image: {digits.images[0].shape}")
```

```
data shape of images: (1797, 8, 8)  
data shape of each image: (8, 8)
```

- 多类别分类问题练习数据集
- 数据类型: Python dictionary
- 1797个手写数字图形数据
- 共10种类别: 0~9
- 每个数字: 8×8 矩阵 = 64个特征

灰度图像：二维数组(2D array)表示图片

8×8矩阵



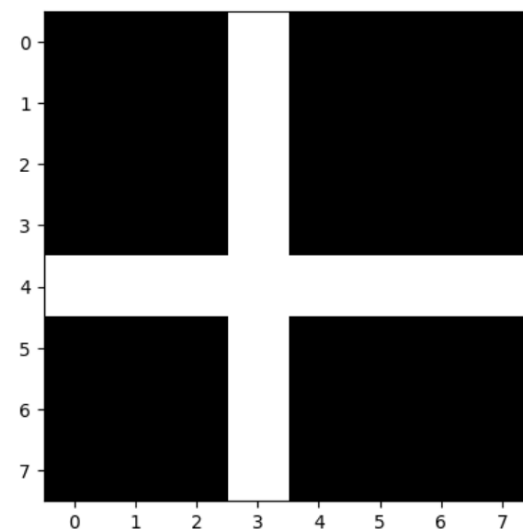
matrix representing an image:

```
[[ 0.  0.  5. 13.  9.  1.  0.  0.]  
 [ 0.  0. 13. 15. 10. 15.  5.  0.]  
 [ 0.  3. 15.  2.  0. 11.  8.  0.]  
 [ 0.  4. 12.  0.  0.  8.  8.  0.]  
 [ 0.  5.  8.  0.  0.  9.  8.  0.]  
 [ 0.  4. 11.  0.  1. 12.  7.  0.]  
 [ 0.  2. 14.  5. 10. 12.  0.  0.]  
 [ 0.  0.  6. 13. 10.  0.  0.  0.]]
```

像素值：通常0~225；本数据集为0~16；
0= '黑色' ， 16 = '白色'

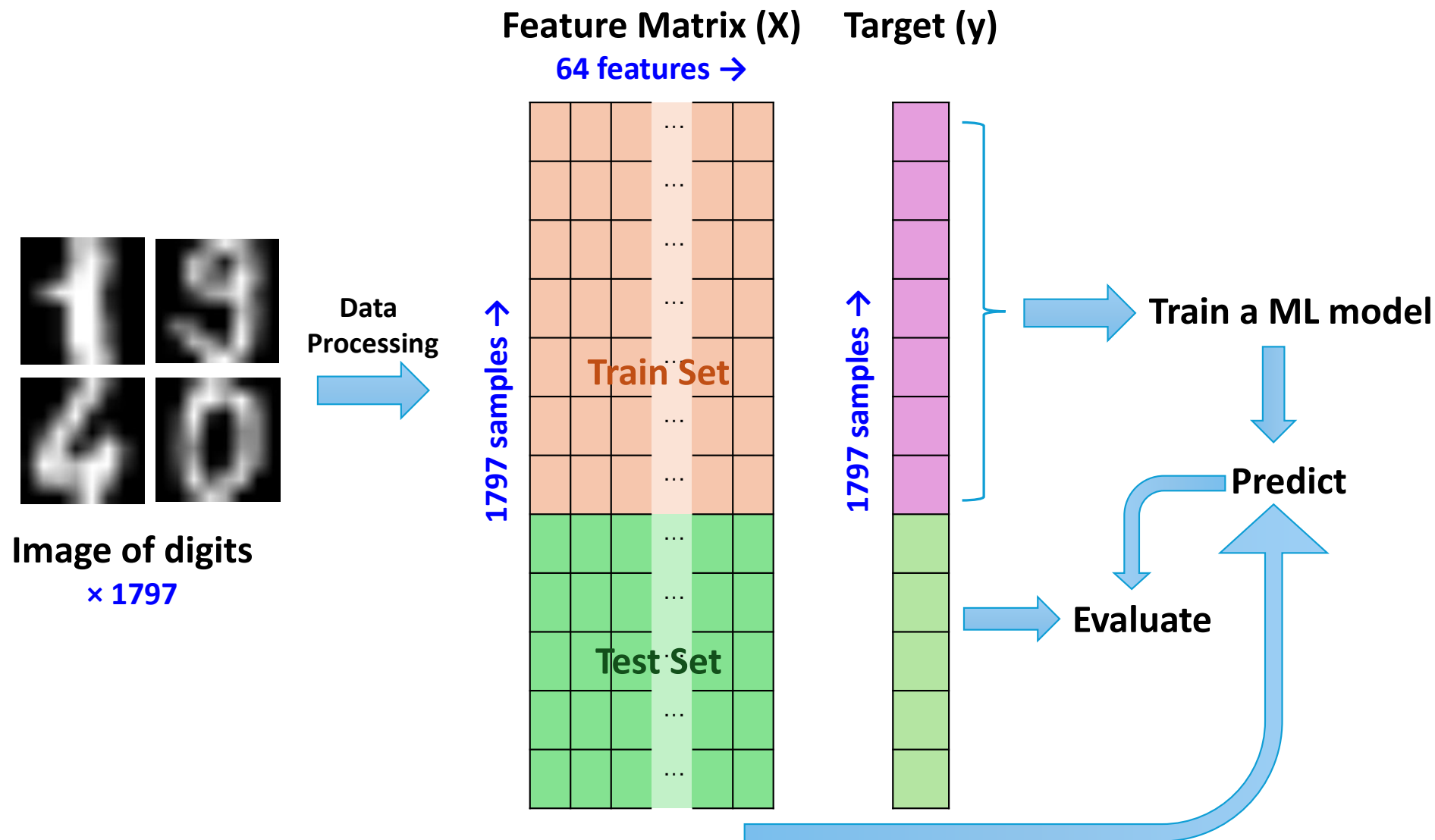
```
plt.imshow(digits.images[0],  
            cmap="gray")
```

练习：自己设置一个8x8二维
数组 newImg显示 “二”



示例， “十”

手写数字识别分类的机器学习流程



数据处理

```
# X: input data (features) , y: targets
digits_X = digits.data
#digits.data.shape = (1797,64), containing flattened image pixel data
digits_y = digits.target

# Split into training (75%) and testing (25%)
digits_X_train, digits_X_test, digits_y_train, digits_y_test = train_test_split(digits_X,
digits_y, test_size=0.25, random_state=42)

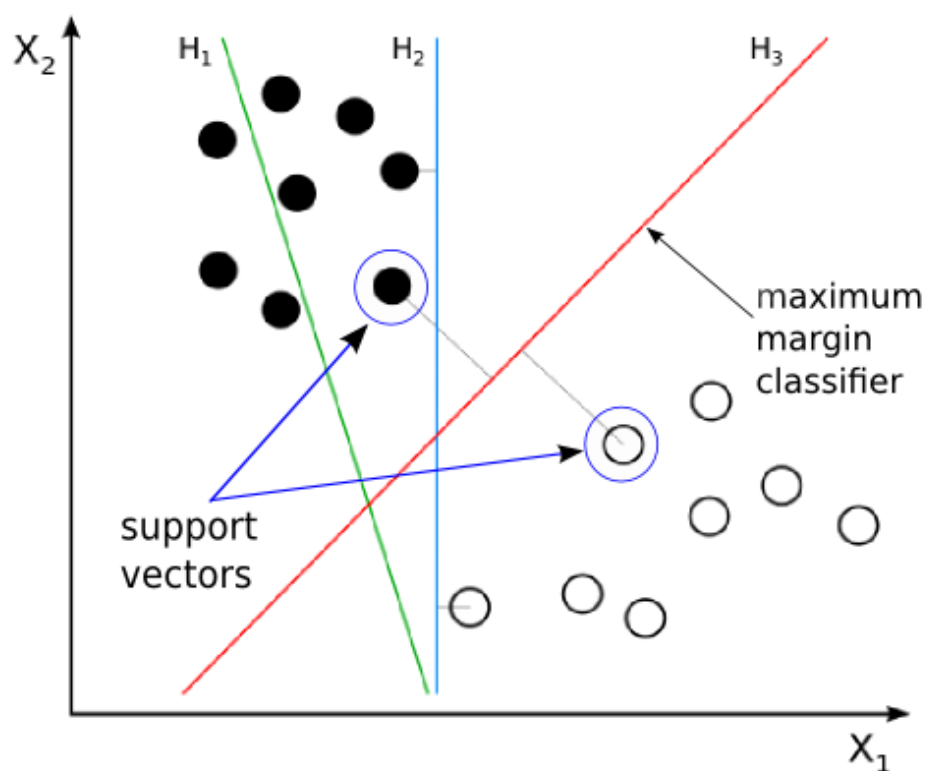
print(f"Training samples: {len(digits_X_train)}")
print(f"Testing samples: {len(digits_X_test)}")
```

```
Training samples: 1347
Testing samples: 450
```

支持向量机分类器

支持向量机 (SVM) 是一种监督学习模型，通过寻找能最大化不同类别数据点间隔的**最优超平面**（即“**最佳边界**”）实现分类。

- **将数据点视为高维空间中的向量**，借助超平面（二维时为直线、三维时为曲面，更高维为复杂形态）完成数据的划分。



- 训练集上“全分对”的直线可能有很多条(如蓝线和红线)。哪一条对未来样本最有把握？SVM 的回答：选择离各类样本都尽可能远的边界。
- 支持向量可理解为真正“撑住”边界的样本（深蓝圈的点）。
- 由最大间隔是问为了让最危险的样本也尽量安全，远离边界的点通常不改变最终解。

支持向量机模型训练

```
from sklearn import svm
svc_model = svm.SVC(gamma=0.001, C=100)
svc_model.fit(digits_X_train, digits_y_train)
```

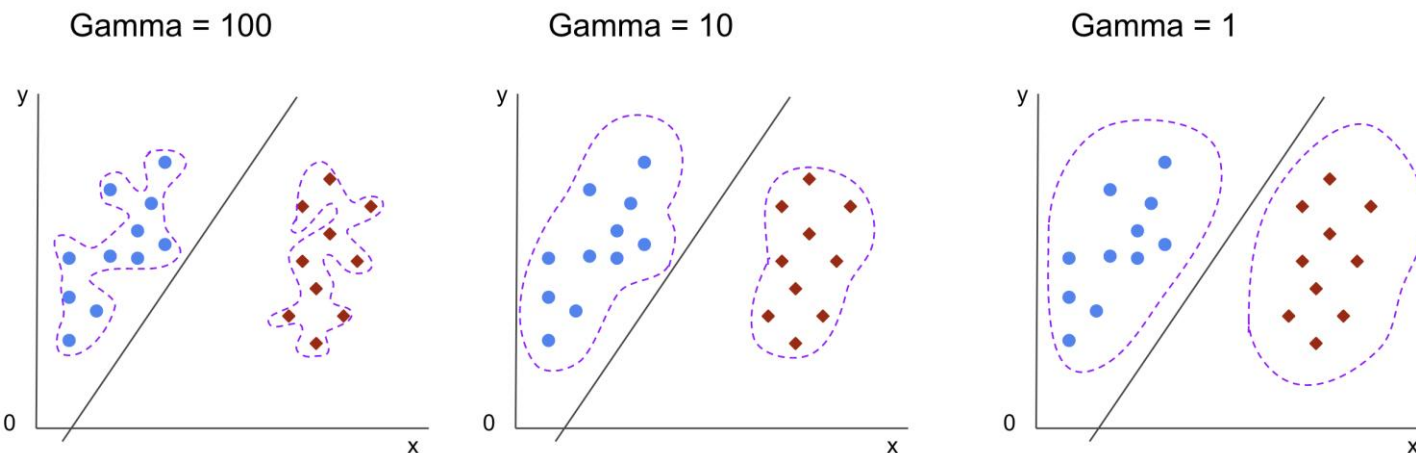
从sklearn函数库中直接导入支持向量机svm算法，训练svm模型需选定一组合适的超参数gamma和C.

参数	较小	较大
C 对训练错误的惩罚程度	更宽容 间隔偏宽，泛化性更强	更严格 更追求贴合训练集
Gamma 单个样本的影响范围	看得远 边界更平滑	看得近 边界更复杂
风险	欠拟合 模式过于简单	过拟合 可能追随噪声

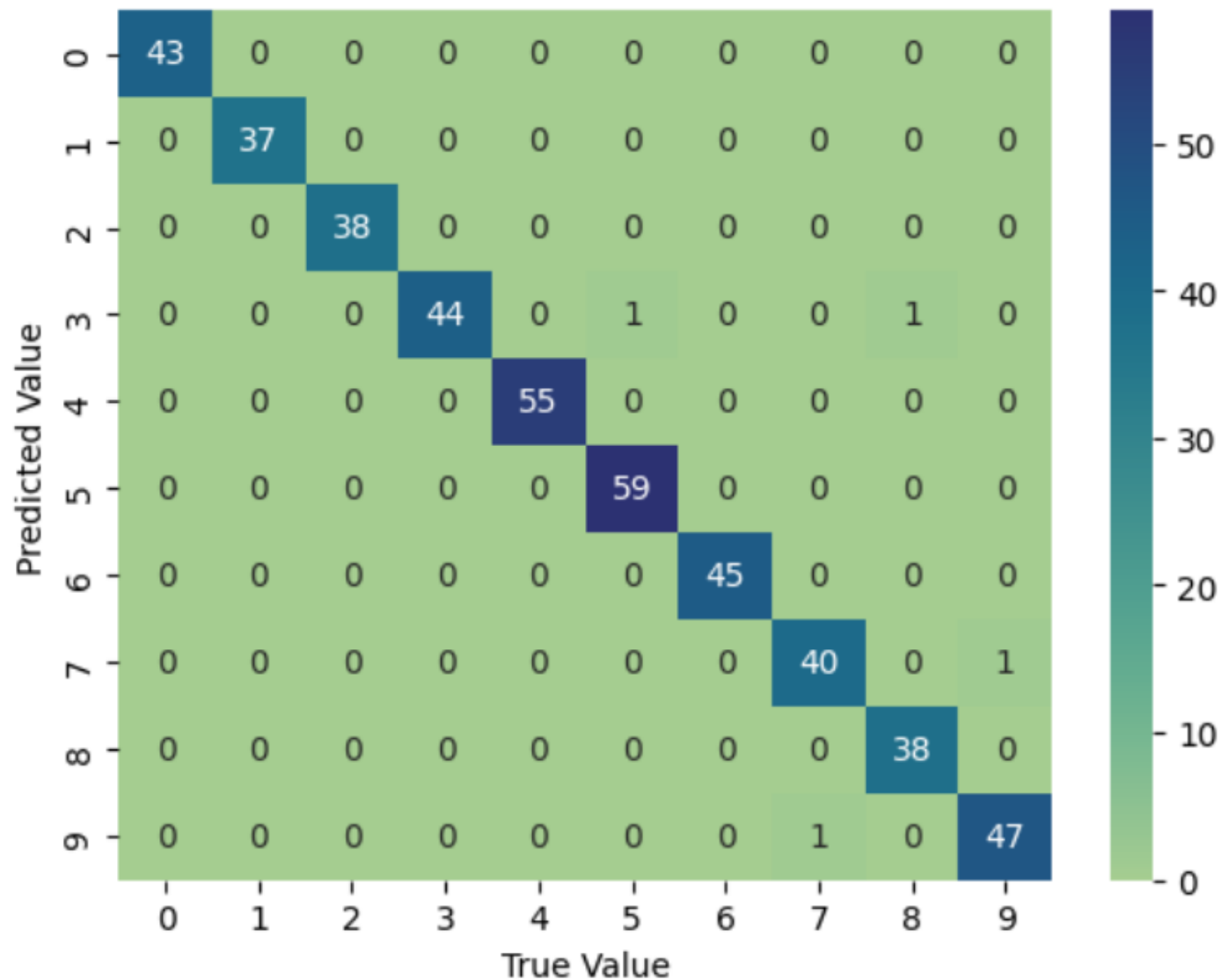
支持向量机模型训练

```
from sklearn import svm
svc_model = svm.SVC(gamma=0.001, C=100)
svc_model.fit(digits_X_train, digits_y_train)
```

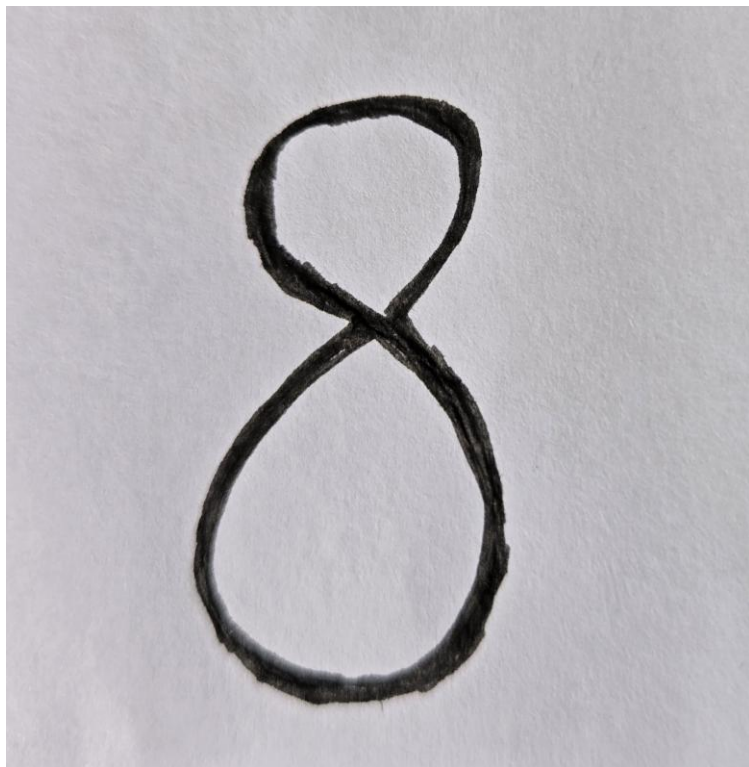
从sklearn函数库中直接导入支持向量机svm算法，训练svm模型需选定一组合适的超参数gamma和C。



分类模型评估：混淆矩阵



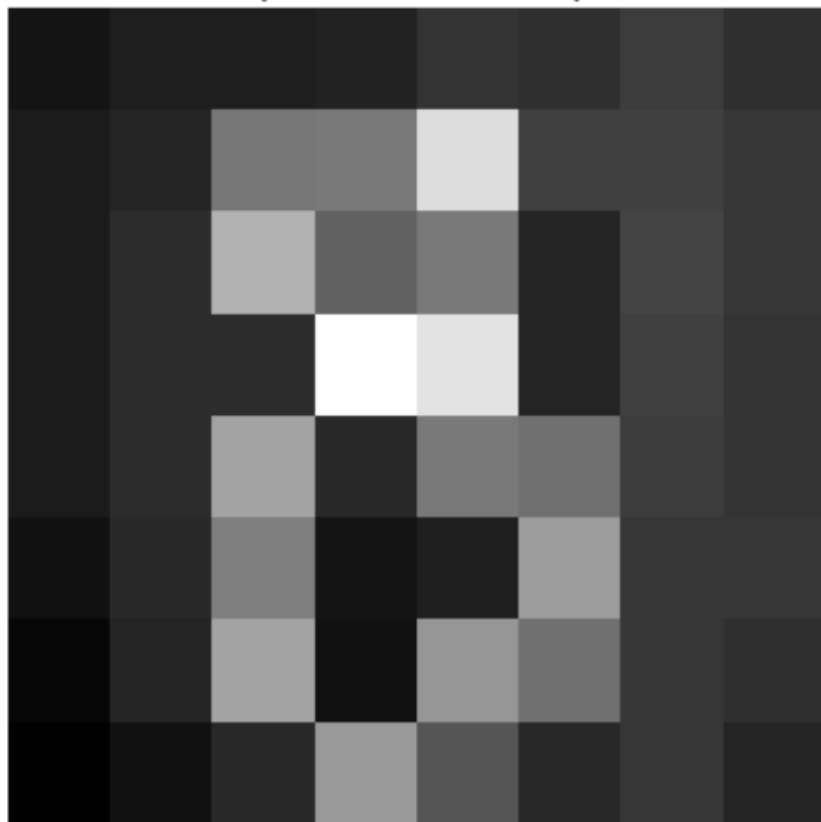
模型预测：预测全新数据（自己制作）



```
my_img = cv2.imread('./img/digit8_photo.jpg') #insert your  
image file path  
my_img_rgb=cv2.cvtColor(my_img, cv2.COLOR_BGR2RGB)  
plt.imshow(my_img_rgb) # use matplotlib imshow to show image  
plt.axis('off') #turn off axis  
plt.show()
```

模型预测：预测全新数据（自己制作）

Preprocessed 8x8 Input



```
def preprocess_image_for_digit(img_path):  
    # Load in grayscale  
    img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)  
    # Resize to 8x8 after cleaning  
    my_img = cv2.bitwise_not(img) # Invert: background white, digit black  
    my_img = cv2.resize(my_img, (8, 8), interpolation=cv2.INTER_AREA)  
    # Normalize to match the digits dataset (0-16)  
    my_img = my_img / 255.0 * 16  
    my_img = my_img.astype(np.float64)  
    # Flatten to shape (1, 64) as model expects  
    my_img_flat = my_img.reshape(1, -1)  
    return my_img_flat, my_img
```

```
img_path = 'digit8_photo.jpg'  
input_flat, input_8x8 = preprocess_image_for_digit(img_path)  
  
# Visualize the preprocessed image  
plt.imshow(input_8x8, cmap='gray')
```

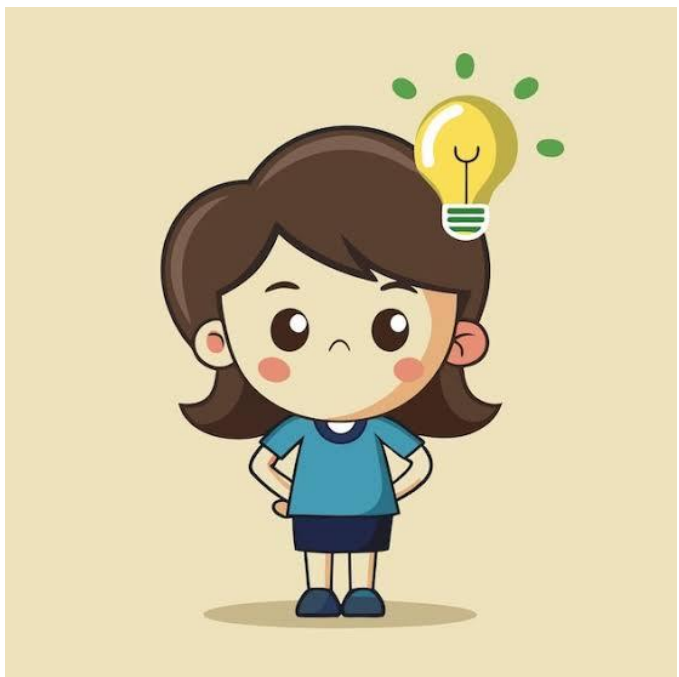

TODO:

1. 每人自己手写数字，拍照，上传
2. 使用同学的照片，用自己训练的支持向量机模型进行预测

思考：拍照的图片其实是黑白图片，转化为 $8*8$ 的黑白图片以后背景不是纯黑色，而是灰色，会影响SVM模型的预测准确度，怎么解决？

灰色背景的解决方案（去年上课的同学提供的思路）

- ✓ 对图片的每个像素进行处理，规定亮度小于某个阈值全部抹0（变为黑色），消除背景干扰。
- ✓ 用“扫描全能王”等类似应用对其进行扫描降噪。

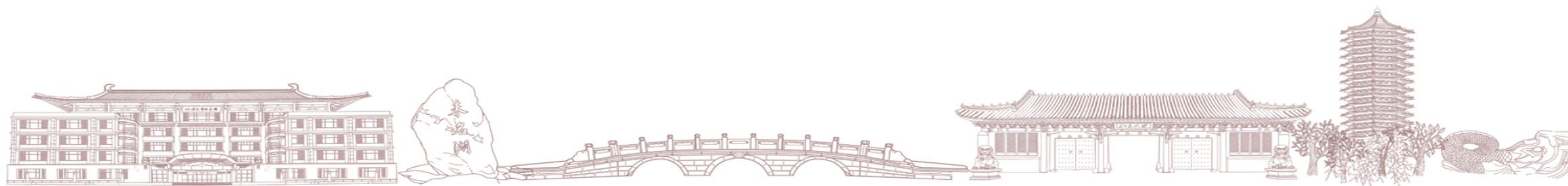




实验二、深度学习在图像分析中的应用

Lab 2: The Application of Deep Learning in Image Analysis

Jupyter notebook: lab2_intro2DL.ipynb

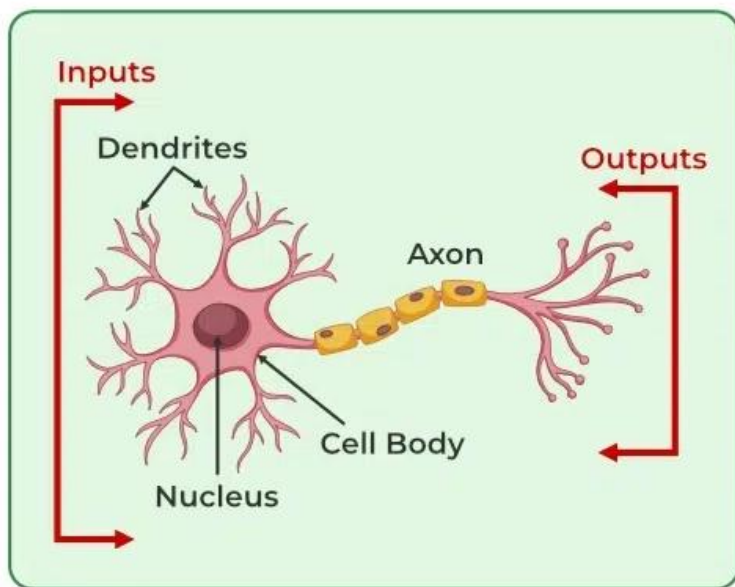


深度学习 Deep Learning

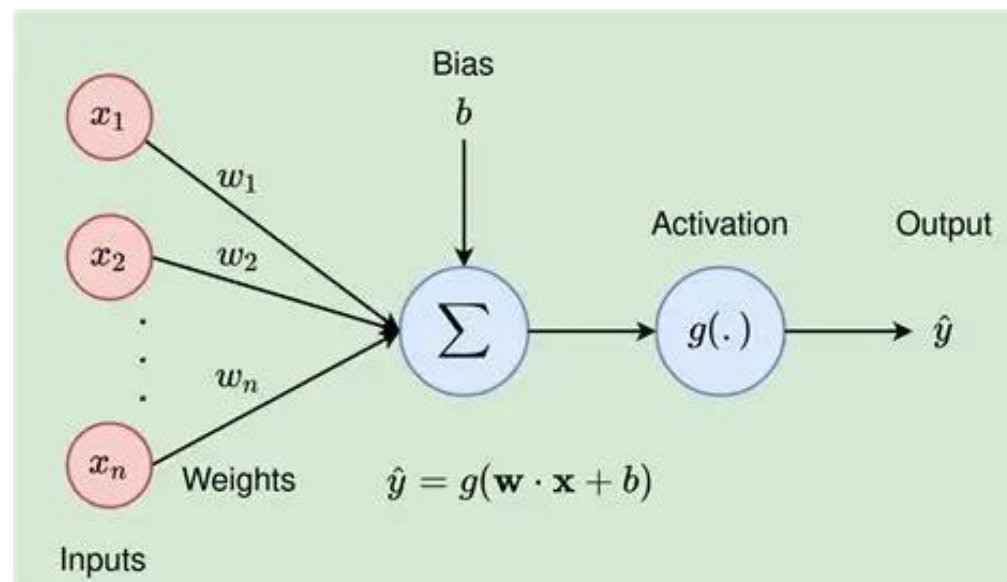
深度学习：机器学习的一个分支，通过构建具有很多**层结构**的模型算法，让计算机自动从大量数据中学习特征和模式。

神经网络：模拟生物神经元结构构建的数学模型，由大量神经元（节点）相互连接组成。每个神经元接收输入，进行计算后产生输出，并传递给其他神经元。。

两者联系：深度学习以神经网络为基础，通常使用包含多个隐藏层的深度神经网络。神经网络为深度学习提供了模型架构，通过优化算法等方式方式，在复杂任务上表现卓越。



神经元

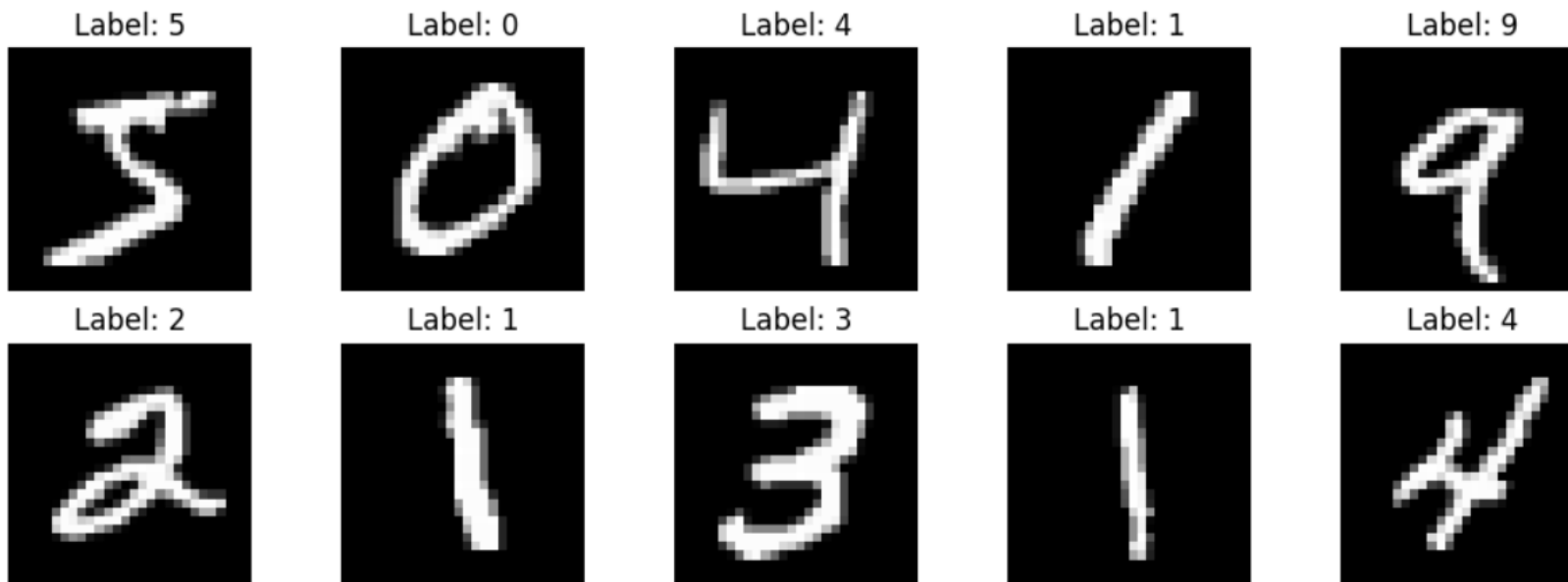


机器学习神经元

Task 1: 多层感知机MNIST数据集分类

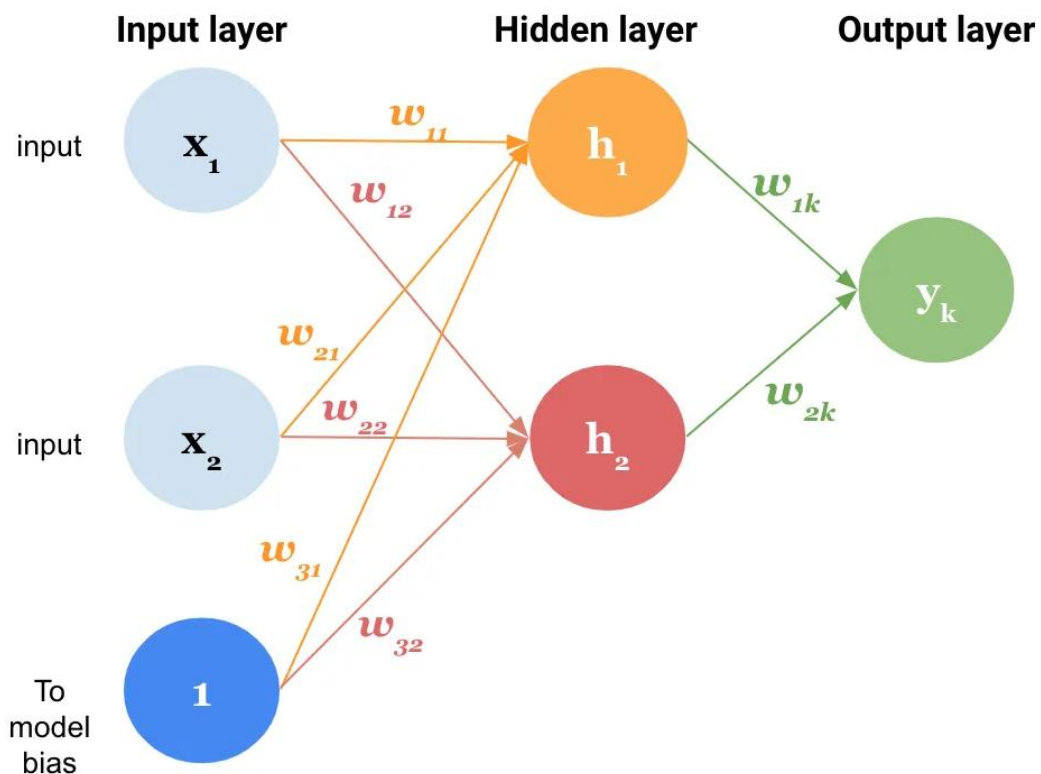
- **MNIST** (Modified National Institute of Standards and Technology) 数据集
- 7万个手写数字灰度图片：6万训练，1万测试
- 共10种类别：0~9
- 每个图片：28×28矩阵=784个像素（特征）
- 每个像素点：0~225；0= ‘黑色’ ， 225 = ‘白色’

MNIST Sample Visualization



多层感知机 Multilayer Perceptron (MLP)

多层感知机 (MLP) 是一种神经网络结构，由多层向前传播的神经网络函数构成，该模型结构主要包含**输入层**、**隐藏层**和**输出层**。



左图例子显示：(a) 一个具有两个输入节点和一个偏置节点（截距项）的输入层；
(b) 一个具有两个节点的隐藏层；
(c) 一个具有一个节点的输出层。

这个模型中共有几个参数？

- 隐藏层的每个神经元和输入层的每一个节点通过参数 w 连接（全连接层 fc）。

$$\begin{aligned} h_1 &= g(w_1^T x + b_1) \\ &= g(w_{11}x_1 + w_{21}x_2 + w_{31}) \end{aligned}$$

- 参数计算：

$$\text{fc1} = 2 \times (2 + 1) = 6; \quad \text{fc2} = 1 \times (2 + 1) = 3$$

共9个参数

MNIST数据集加载

```
# Load the raw MNIST dataset
```

```
mnist_train_raw = torchvision.datasets.MNIST(root='./data', train=True, download=True, transform=None)
mnist_test_raw = torchvision.datasets.MNIST(root='./data', train=False, download=True, transform=None)
```

```
#explore the dataset
```

```
print(f"the data type of mnist_train_raw is: {type(mnist_train_raw)}")
```

```
print(f"Training set size: {len(mnist_train_raw)}")
```

```
print(f"Testing set size: {len(mnist_test_raw)}")
```

```
#TODO set to a certain index to view the data
```

```
n = 0
```

```
print(f"the type nth element: {type(mnist_train_raw[n])}")
```

```
print(f"the nth element in the raw training set: {mnist_train_raw[n]}")
```

Python

```
the data type of mnist_train_raw is: <class 'torchvision.datasets.mnist.MNIST'>
```

```
Training set size: 60000
```

```
Testing set size: 10000
```

```
the type nth element: <class 'tuple'>
```

```
the nth element in the raw training set: (<PIL.Image.Image image mode=L size=28x28 at 0x289743F9A90>, 5)
```

MNIST数据集查看

```
#save the PIL image object and corresponding lable
#for nth element in two seperate variables
image_n,label_n = mnist_train_raw[n]
print(f"the image format, size and mode for nth image is: {image_n.format},{image_n.size},{image_n.mode}")
print(f"the label for nth image is: {label_n}")
image_n #show image_n
```

Python

```
the image format, size and mode for nth image is: None,(28, 28),L
the label for nth image is: 5
```



#使用getpixel()查看某像素点的值

```
pixel = image_n.getpixel((x, y))
```


张量

Tensor Order	Example	Shape	Description
0D	3.14	()	Scalar
1D	[1, 2, 3]	(3,)	Vector
2D	[[1, 2], [3, 4]]	(2, 2)	Matrix
3D+	[[[...]]]	(n, m, k)	Tensor (3D or more)

np.array vs torch.tensor

- Tensor与NumPy Array 在数据结构上相似
- Tensor的计算图优化、自动微分以及与深度学习框架的完美集成使其在深度学习中具有明显优势
- Torch.tensor 在Pytorch框架中使用方便
- Tensor更适合大规模并行计算和GPU加速

训练神经网络需要把图像对象转化为张量

```
# Data preprocessing
```

```
transform = transforms.Compose([  
    transforms.ToTensor(),  
    transforms.Normalize((0.1307,), (0.3081,)) # MNIST mean and std  
])
```

```
# Load normalized data as tensors
```

```
mnist_train = torchvision.datasets.MNIST(root='./data', train=True,  
download=True, transform=transform)  
mnist_test = torchvision.datasets.MNIST(root='./data', train=False,  
transform=transform)
```

torchvision.transforms ()

```
print(f"the nth element in the raw training set: {mnist_train_raw[n]}")
print(f"the nth element in the transformed training set:\n {mnist_train[n]}")
```

```
the nth element in the raw training set: (<PIL.Image.Image image mode=L size=28x28 at 0x289745A2730>, 5)
```

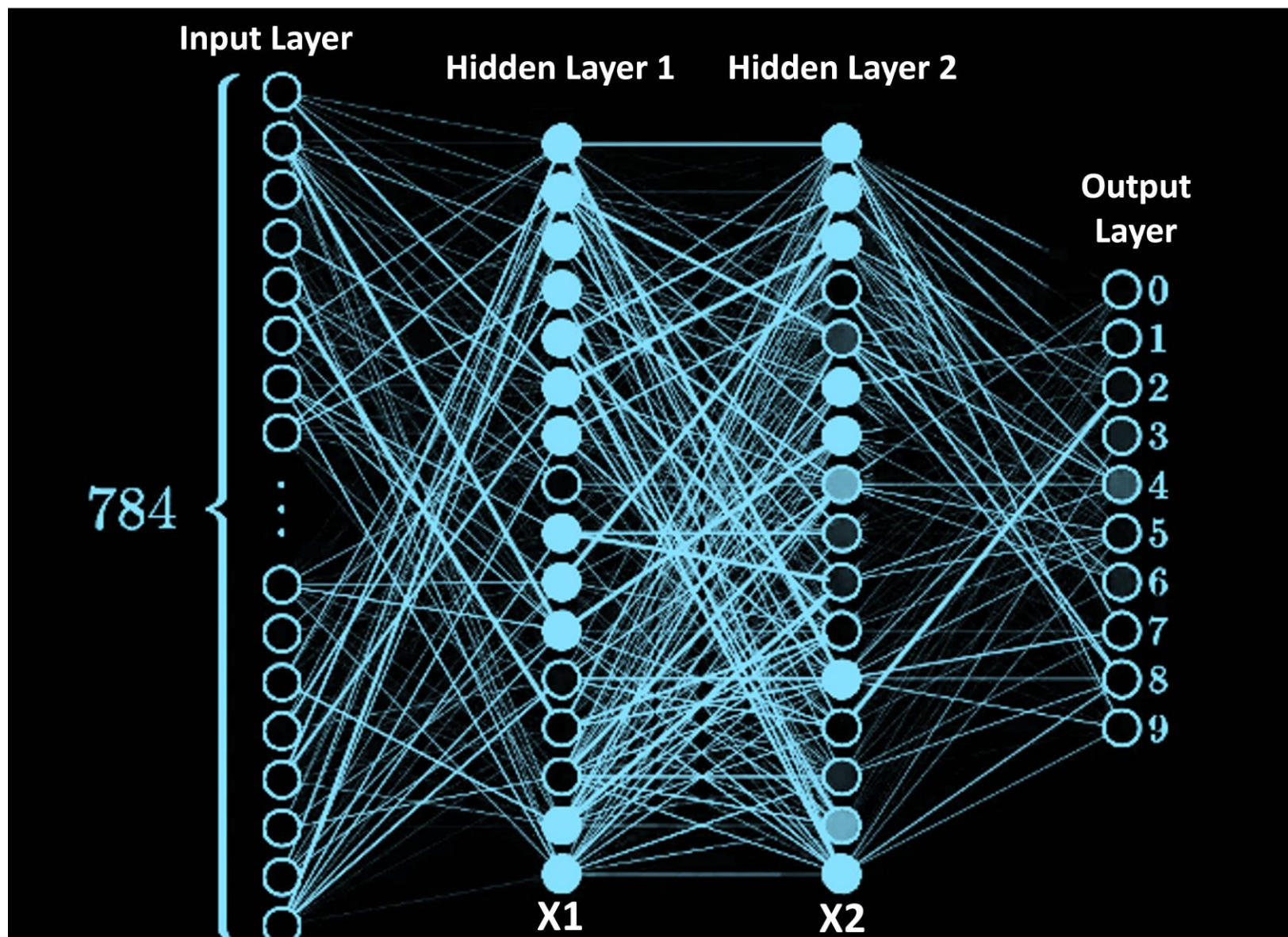
the n th element in the transformed training set:

[illegible]

• • •

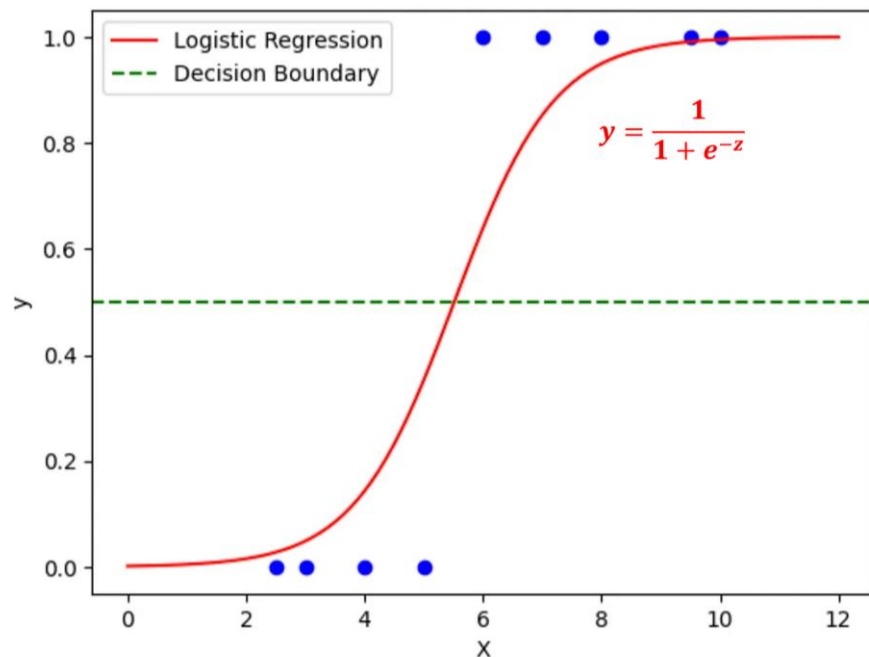
$$\begin{bmatrix} -0.4242, & -0.4242, & -0.4242, & -0.4242, & -0.4242, & -0.4242, & -0.4242, \\ -0.4242, & -0.4242, & -0.4242, & -0.4242, & -0.4242, & -0.4242, & -0.4242. \end{bmatrix}$$

多层感知机 Multilayer Perceptron (MLP)

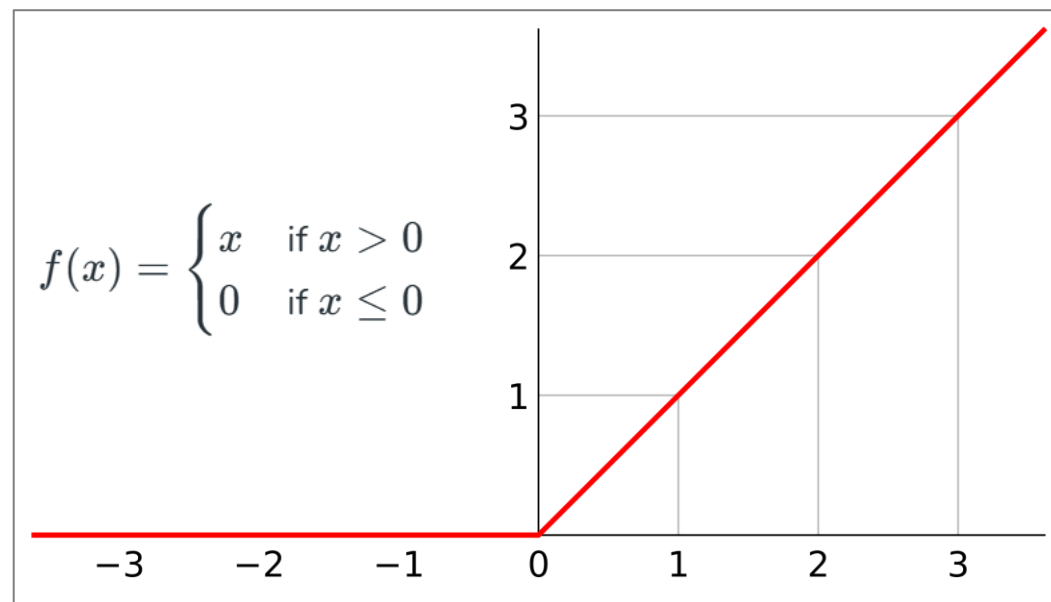


激活函数 Activation Function

Sigmoid 函数

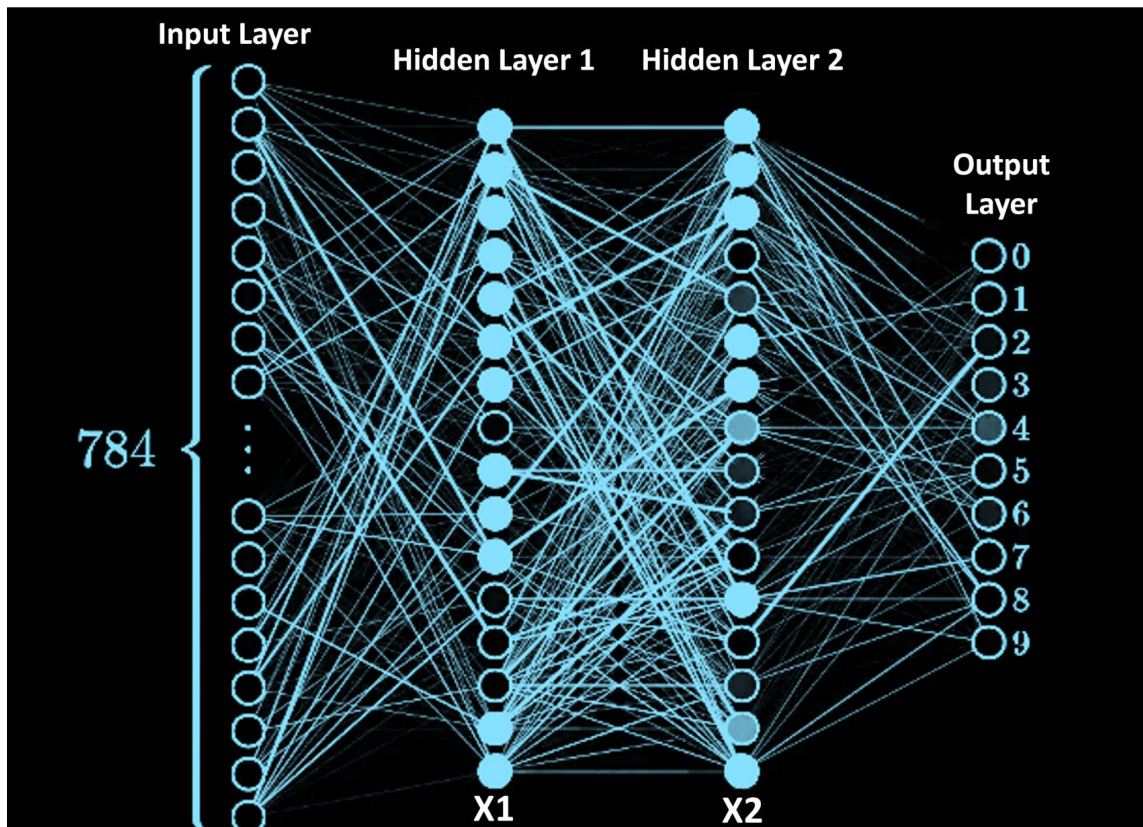


Relu 函数



在每个全连接层（如 fc1）之后，在每个隐藏层后引入**非线性函数**来**激活**神经网络。常见的非线性函数有ReLU和sigmoid函数。

搭建多层感知机代码实现



1. 定义一个类 `SimpleMLP(nn.Module)`
继承 `torch.nn.model()`类的结构

2. 定义模型结构

```
self.flatten = nn.Flatten()  
self.fc1 = nn.Linear(784, 128)  
self.fc2 = nn.Linear(128, 64)  
self.fc3 = nn.Linear(64, 10)  
self.dropout = nn.Dropout(0.2)
```

The exact parameter count for this SimpleMLP is calculated as:

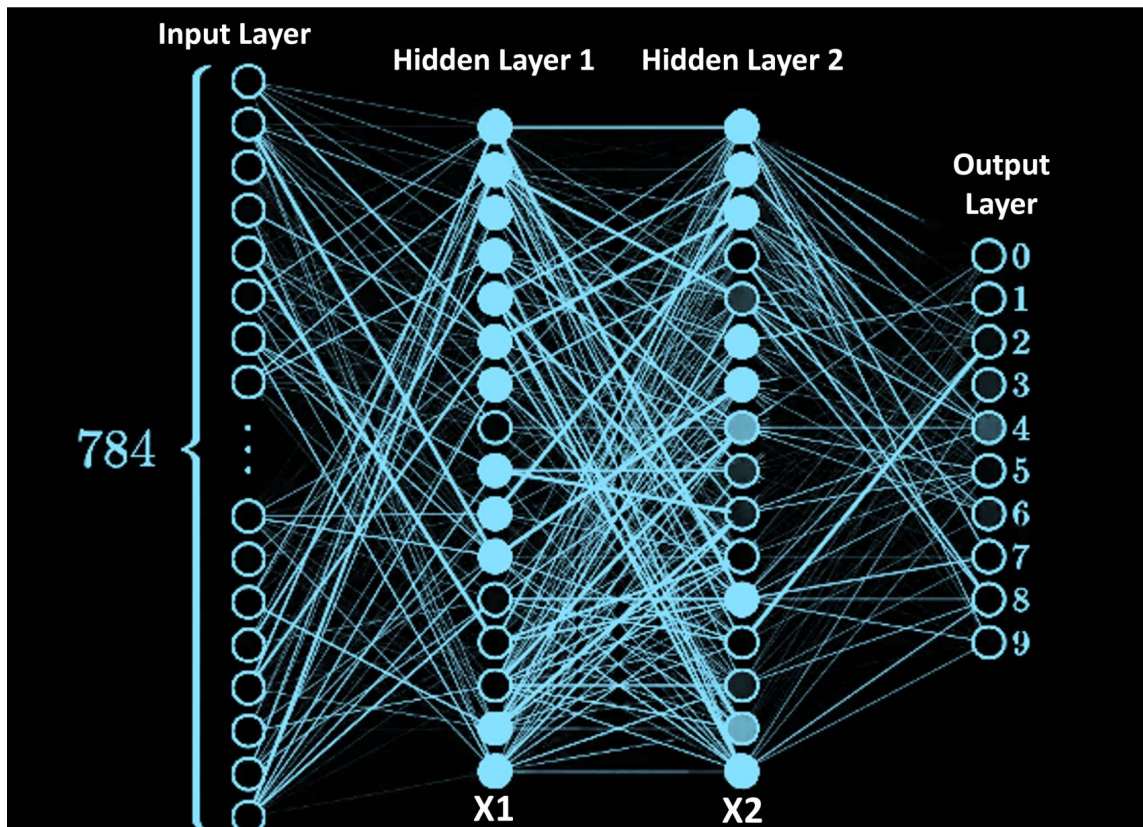
fc1 layer: $784 \times 128 + 128 = 100,480$

fc2 layer: $128 \times 64 + 64 = 8,256$

fc3 layer: $64 \times 10 + 10 = 650$

Total parameters = $100,480 + 8,256 + 650 = 109,386$

搭建多层感知机代码实现



3.定义向前传播

```
x = self.flatten(x)
x = torch.relu(self.fc1(x))
x = self.dropout(x)
x = torch.relu(self.fc2(x))
x = self.dropout(x)
return self.fc3(x)
```


损失函数

损失函数 (Loss Function) 是深度学习中衡量模型预测值与真实值差异的核心工具，其优化目标是通过梯度下降等算法最小化损失值。常见的损失函数可分为以下几类：

$$L = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

- **均方误差损失 (MSE, Mean Squared Error)**

- 适用于回归任务，计算预测值与真实值的平方差均值。
- 特点：对异常值敏感，输出为连续值时常用。

- **平均绝对误差损失 (MAE, Mean Absolute Error)**

- 回归任务中另一种鲁棒性更强的损失函数，计算绝对误差均值。公式
- 特点：对异常值不敏感，但梯度更新不如MSE稳定。

$$L = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

- **交叉熵损失 (Cross-Entropy Loss)**

分类任务的核心损失函数，尤其适用于二分类和多分类问题

- **自定义损失函数**

针对特定任务设计（如目标检测中的Focal Loss解决类别不平衡问题）

交叉熵损失函数

- **二分类交叉熵 (Binary Cross-Entropy)**

$$L = \frac{1}{N} \sum_i L_i = \frac{1}{N} \sum_i -[y_i \cdot \log(p_i) + (1 - y_i) \cdot \log(1 - p_i)]$$

- N 样本数量
- y_i 样本 i 的真实标签(0 or 1),
- p_i 模型预测样本 i 为正类 (1) 的概率

- **多分类交叉熵 (Categorical Cross-Entropy)**

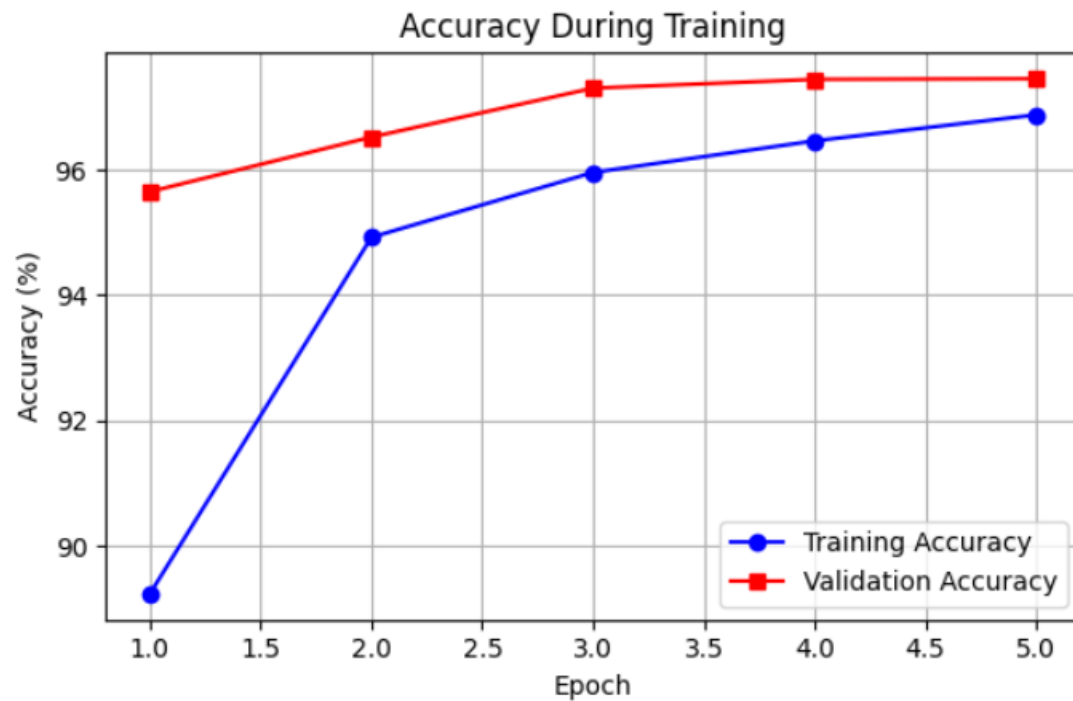
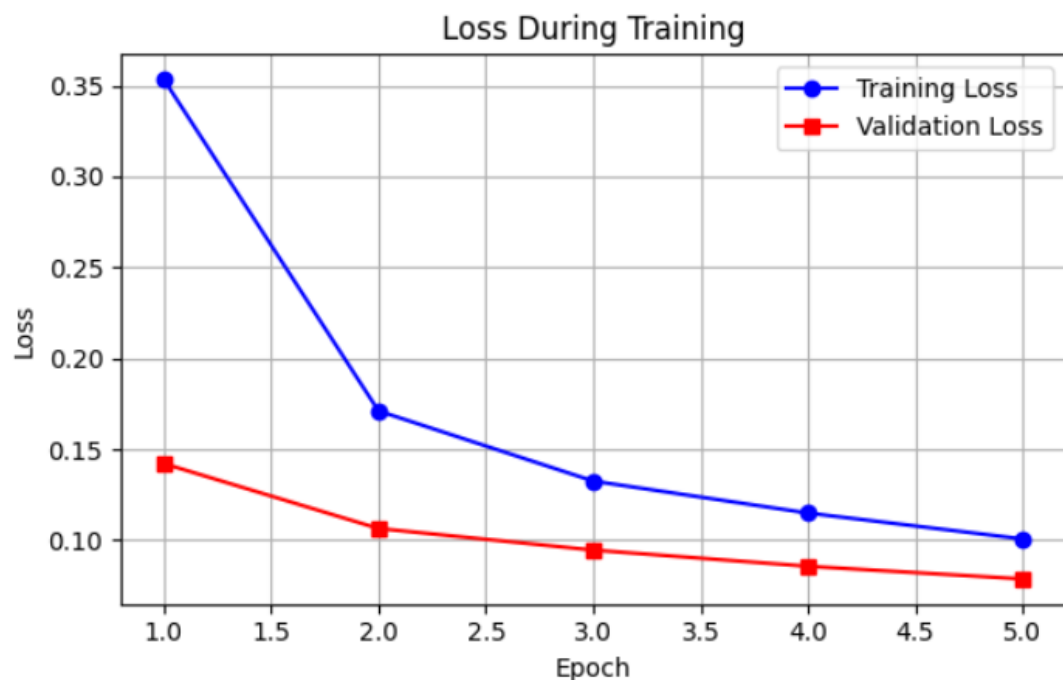
$$L = \frac{1}{N} \sum_i L_i = -\frac{1}{N} \sum_i \sum_{c=1}^M y_{ic} \log(p_{ic})$$

- M 类别数
- y_{ic} 样本 i 的真实类别, 等于 c 取1, 否则取0
- p_{ic} 模型预测样本 i 为类别 c 的概率

模型训练

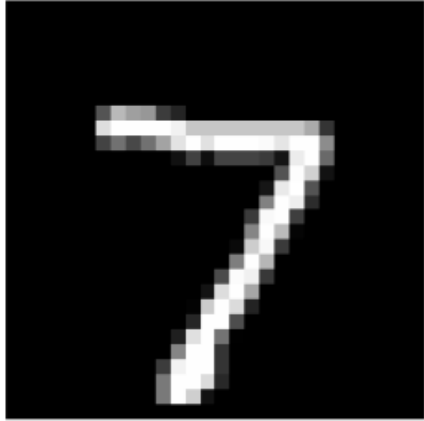
定义

- 损失函数: `nn.CrossEntropyLoss()`
- 优化器: `optim.Adam(model.parameters(), lr=0.001)`



MNIST Test Results (Green=Correct, Red=Incorrect)

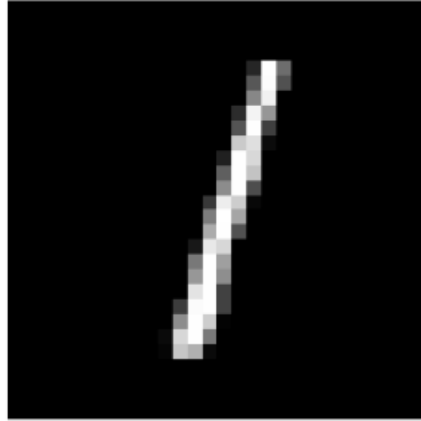
True: 7, Pred: 7



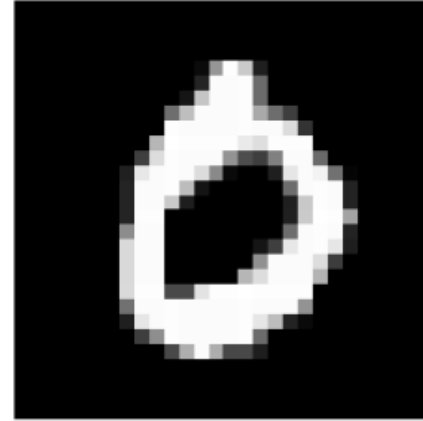
True: 2, Pred: 2



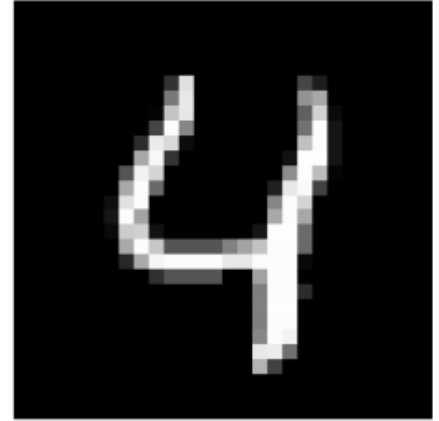
True: 1, Pred: 1



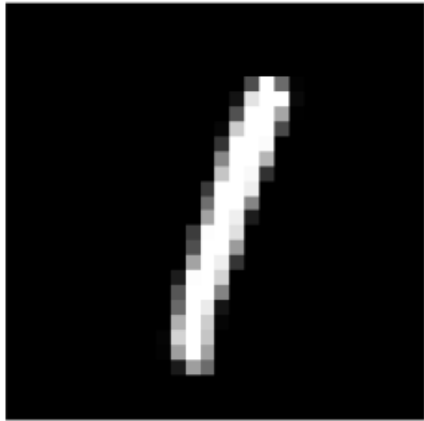
True: 0, Pred: 0



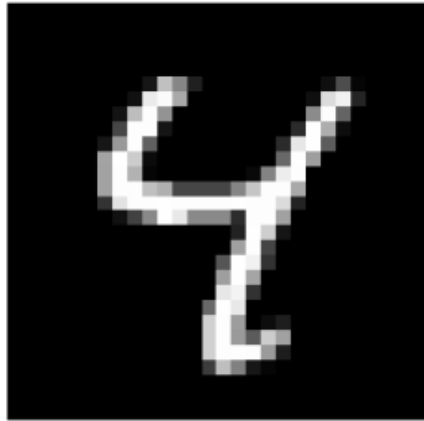
True: 4, Pred: 4



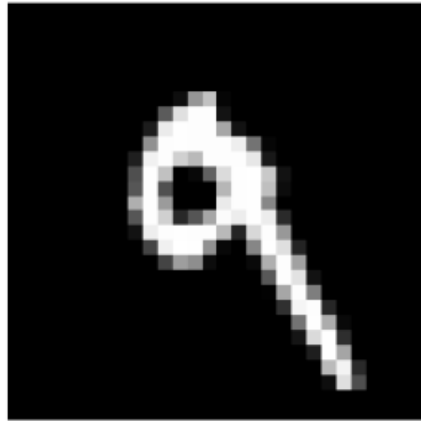
True: 1, Pred: 1



True: 4, Pred: 4



True: 9, Pred: 9



True: 5, Pred: 5



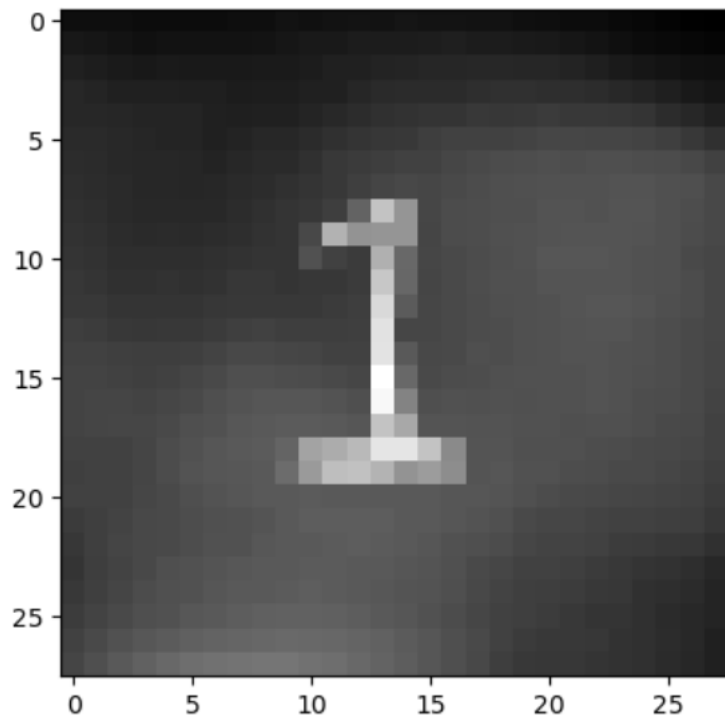
True: 9, Pred: 9



TODO

用自己的图片测试模型

Task 1总结



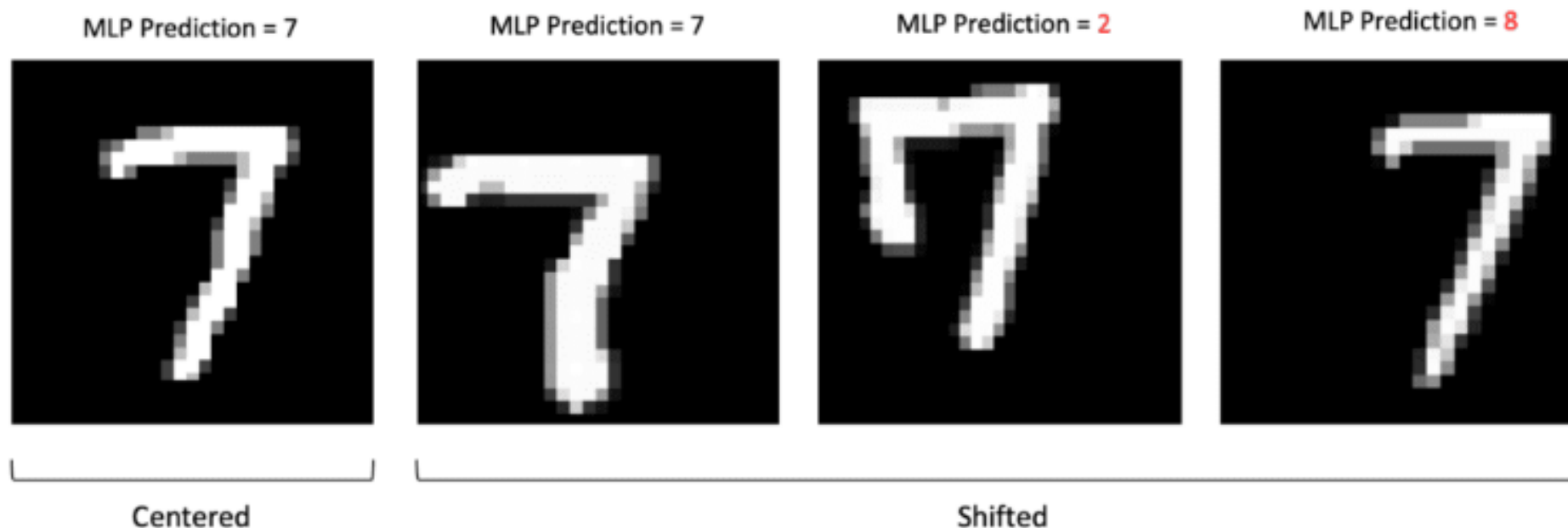
(graph credit to: 谢一鸣)

- 数据集自带测试集: $\text{accuracy} > 96\%$
- 对**全新数据（自制）**的预测:
 - $\text{accuracy} < 50\%$ (粗估)
- 可能原因讨论:
 - 正方形图片 vs 长方形图片 (拉伸导致变形) ?
 - 拍摄照片背景 **≠ 纯白** → 灰度矩阵背景 **≠ 纯黑**
- 如何解决?

Task 2: 应用卷积神经网络 (CNN) 进行图片分类

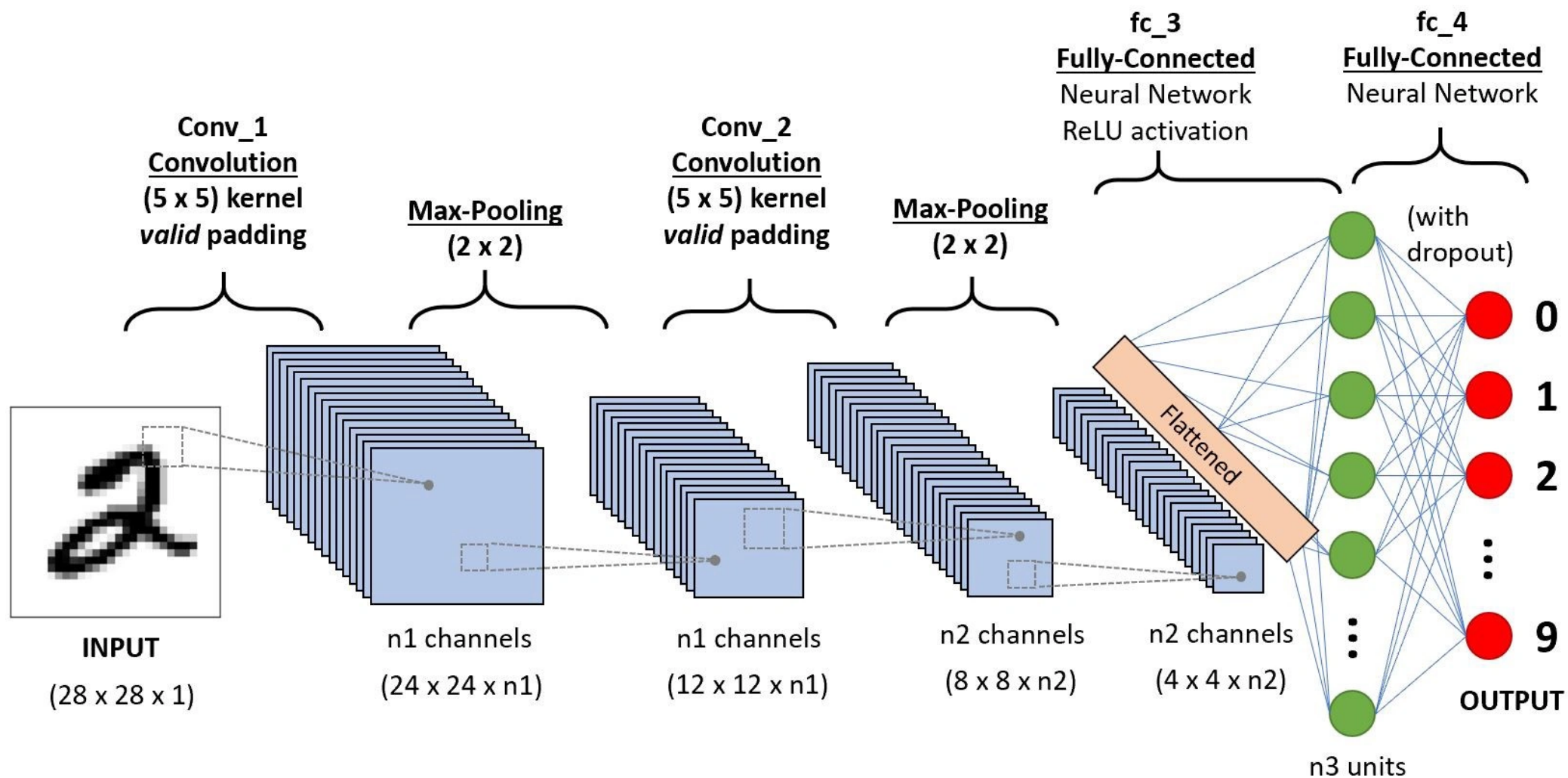
多层感知机(MLP)的缺陷:

- 不具备平移不变性、缩放不变性、旋转不变性。。。
- 有过拟合的倾向 (参数太多)

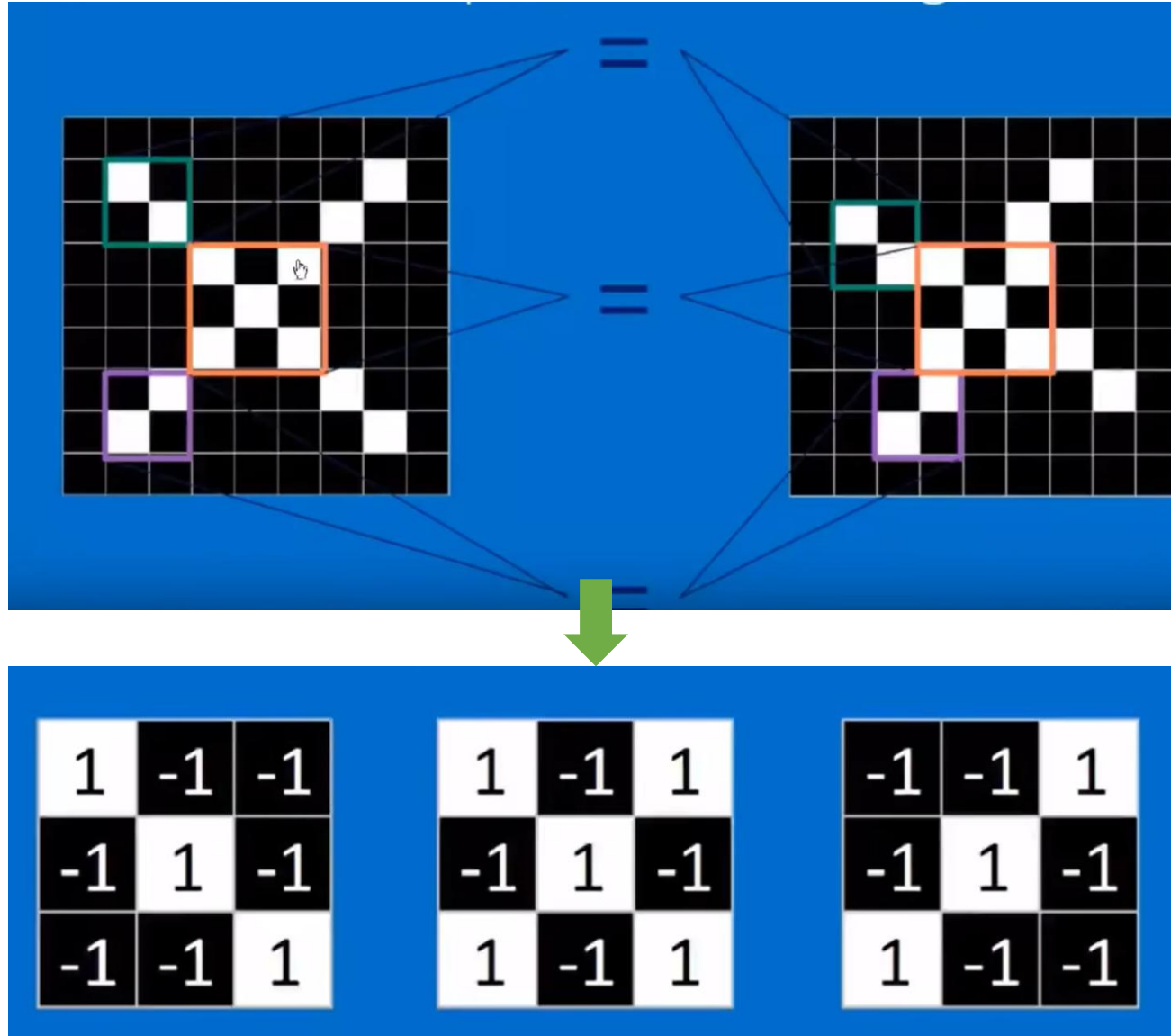


卷积神经网络 (CNN) 的发明就是为了解决这些问题。

卷积神经网络 Convolutional Neural Network (CNN)



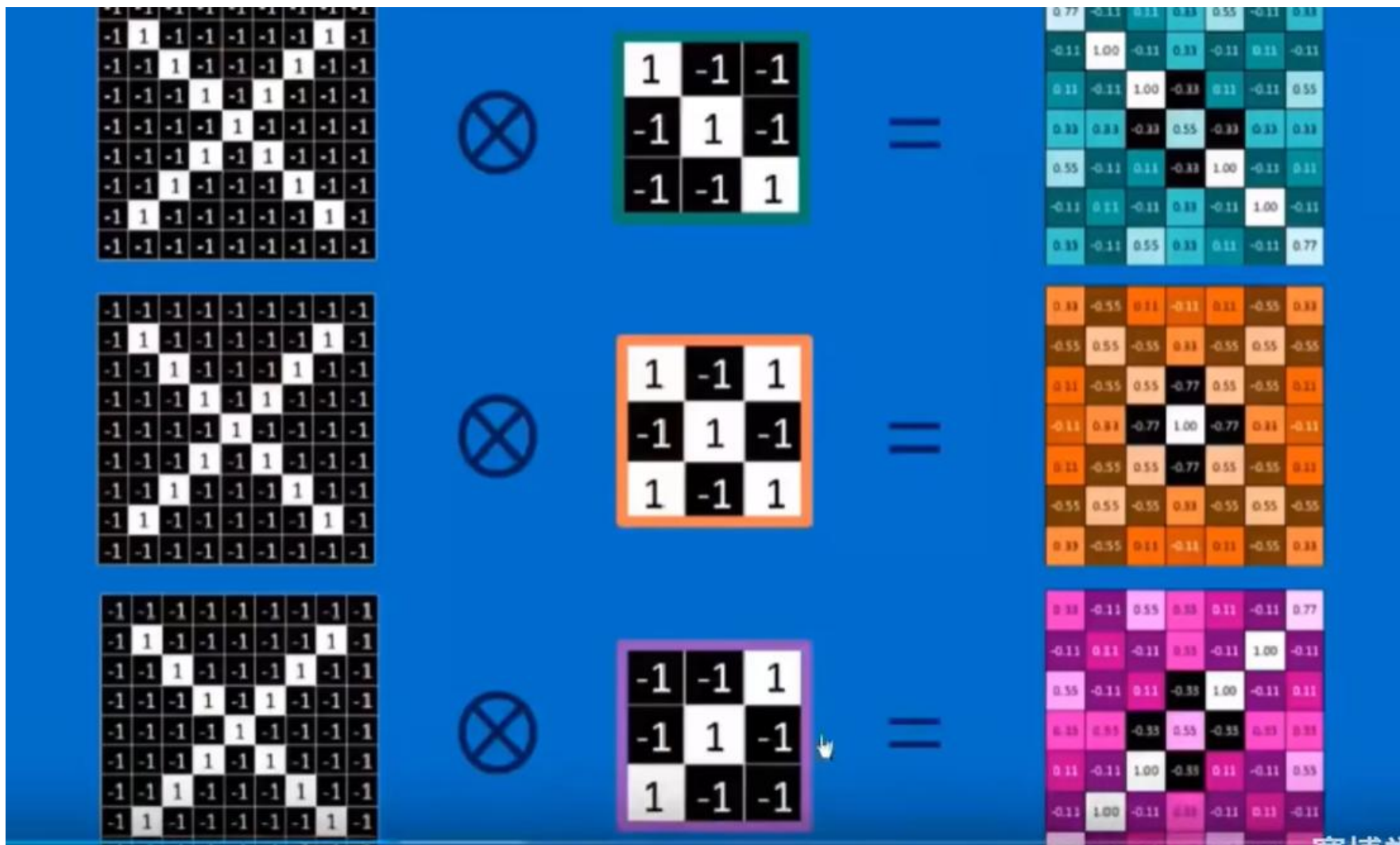
本地特征 local features



核函数
kernel function

卷积运算中的通道

输入通道=1



输出通道=3

核函数的个数 = 卷积层的通道

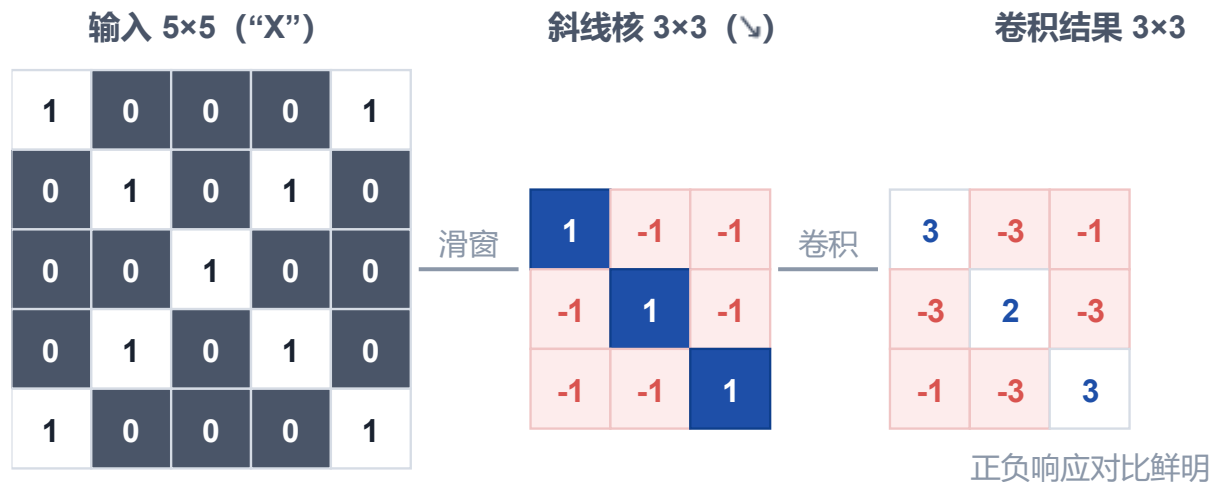
斜线核：↘ 为 1，其余为 -1

这个 3×3 卷积核，只有**主对角线**（左上 → 右下 ↘）上是 1，其余全是 **-1**。

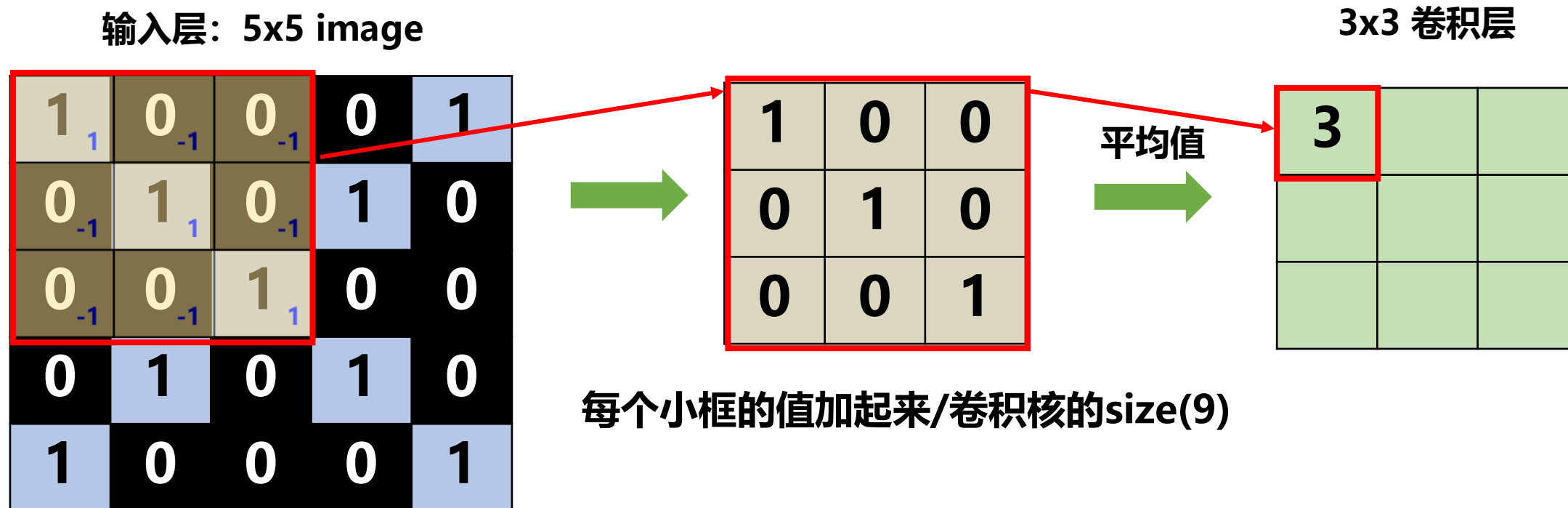
所以它做卷积时，**等价于“↘ 对角线 和 减去 其余 6 个数之和”** ——
匹配给正分，不匹配扣分。

核 = [1 -1 -1; -1 1 -1; -1 -1 1]

输出 = ↘ 对角线和 - 其余之和



卷积运算 – step 1



卷积运算 – step 2

1	0 ₁	0 ₋₁	0 ₋₁	1
0	1 ₋₁	0 ₁	1 ₋₁	0
0	0 ₋₁	1 ₋₁	0 ₁	0
0	1	0	1	0
1	0	0	0	1

输入层: 5x5 image



0	0	0
-1	0	-1
0	-1	0

平均值



3x3 卷积层

3	-3	

卷积运算 – step 3

输入层: 5x5 image

1	0	0 ₁	0 ₋₁	1 ₋₁
0	1	0 ₋₁	1 ₁	0 ₋₁
0	0	1 ₋₁	0 ₋₁	0 ₁
0	1	0	1	0
1	0	0	0	1



0	0	-1
0	1	0
-1	0	0

平均值



3x3 卷积层

3	-3	-1

(此处省略5步。)

卷积运算 – step9

输入层: 5x5 image

1	0	0	0	1
0	1	0	1	0
0	0	1 ₁	0 ₋₁	0 ₋₁
0	1	0 ₋₁	1 ₁	0 ₋₁
1	0	0 ₋₁	0 ₋₁	1 ₁



1	0	0
0	1	0
0	0	1

平均值



3x3 卷积层

3	-3	-1
-3	2	-3
-1	-3	3

逐位置计算

以左上角窗口为例：核与窗口“对应相乘再求和”，等效于“↘ 对角线和 - 其余 6 个数之和”。

卷积结果 3×3

3	-3	-1
-3	2	-3
-1	-3	3

左上角：↘ 3 - 其余 0 = 3

右上角：↘ 1 - 其余 2 = -1

把 9 个输出按位置填回，
即得卷积结果 [3 -3 -1; -3 2 -3; -1 -3 3]

窗口(行,列)	↘和 - 其余和	输出
(0,0)	3 - 0	3
(0,1)	0 - 3	-3
(0,2)	1 - 2	-1
(1,0)	0 - 3	-3
(1,1)	3 - 1	2
(1,2)	0 - 3	-3
(2,0)	1 - 2	-1
(2,1)	0 - 3	-3
(2,2)	3 - 0	3

卷积运算+Relu激活

输入层: 5x5 image

1	0	0	0	1
0	1	0	1	0
0	0	1	0	0
0	1	0	1	0
1	0	0	0	1

卷积核

1	-1	-1
-1	1	-1
-1	-1	1

步长(stride)=1



卷积运算

3x3 卷积层

3	-3	-1
-3	2	-3
-1	-3	3

Relu激活函数



3	0	0
0	2	0
0	0	3

步长越大，采样越稀，输出越小

stride = 1

输出 3×3

3	-3	-1
-3	2	-3
-1	-3	3

窗口每次移动 1 格

stride = 2

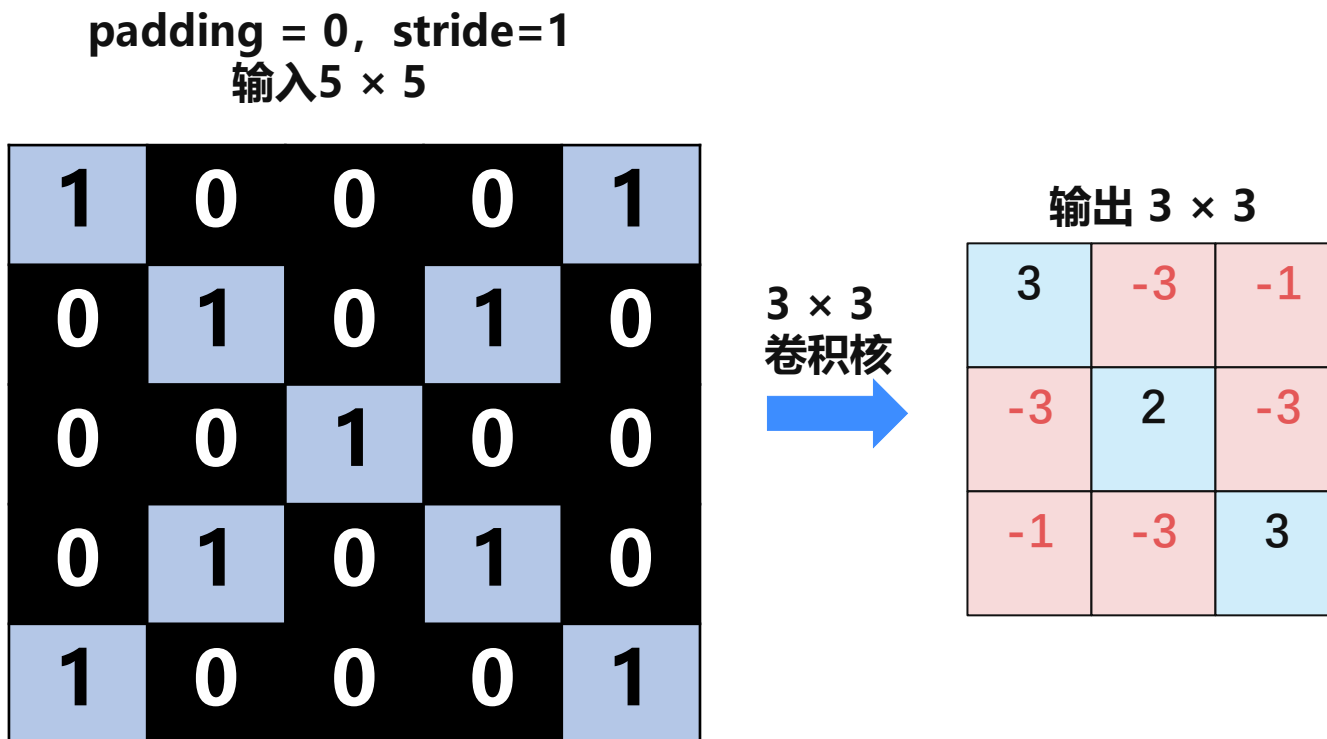
输出 2×2

3	-1
-1	3

窗口每次跨过 2 格

stride $\uparrow$ $\rightarrow$ 计算量 $\downarrow$ ，但可能跳过细节

如果padding=1, 卷积运算之后size变小（越卷越小）



padding = 0: 尺寸缩小

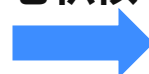
Padding=1 给边界补 0，让边缘也能被充分扫描, 尺寸不变

padding = 1, stride=1
输入 5×5

0	0	0	0	0	0	0
0	1	0	0	0	1	0
0	0	1	0	1	0	0
0	0	0	1	0	0	0
0	0	1	0	1	0	0
0	1	0	0	0	1	0
0	0	0	0	0	0	0

padding = 1: 7×7 输入区
灰色外圈是补入的 0

3×3
卷积核



输出 5×5

2	-2	0	-2	0
-2	3	-3	-1	-2
0	-3	2	-3	0
-2	-1	-3	3	-2
0	-2	0	-2	2

padding = 1: stride=1 时输出尺寸保持输入尺寸 5×5

同一个卷积核，padding 改变边界响应与输出尺寸

5 × 5 输出特征图

2	-2	0	-2	0
-2	3	-3	-1	-2
0	-3	2	-3	0
-2	-1	-3	3	-2
0	-2	0	-2	2

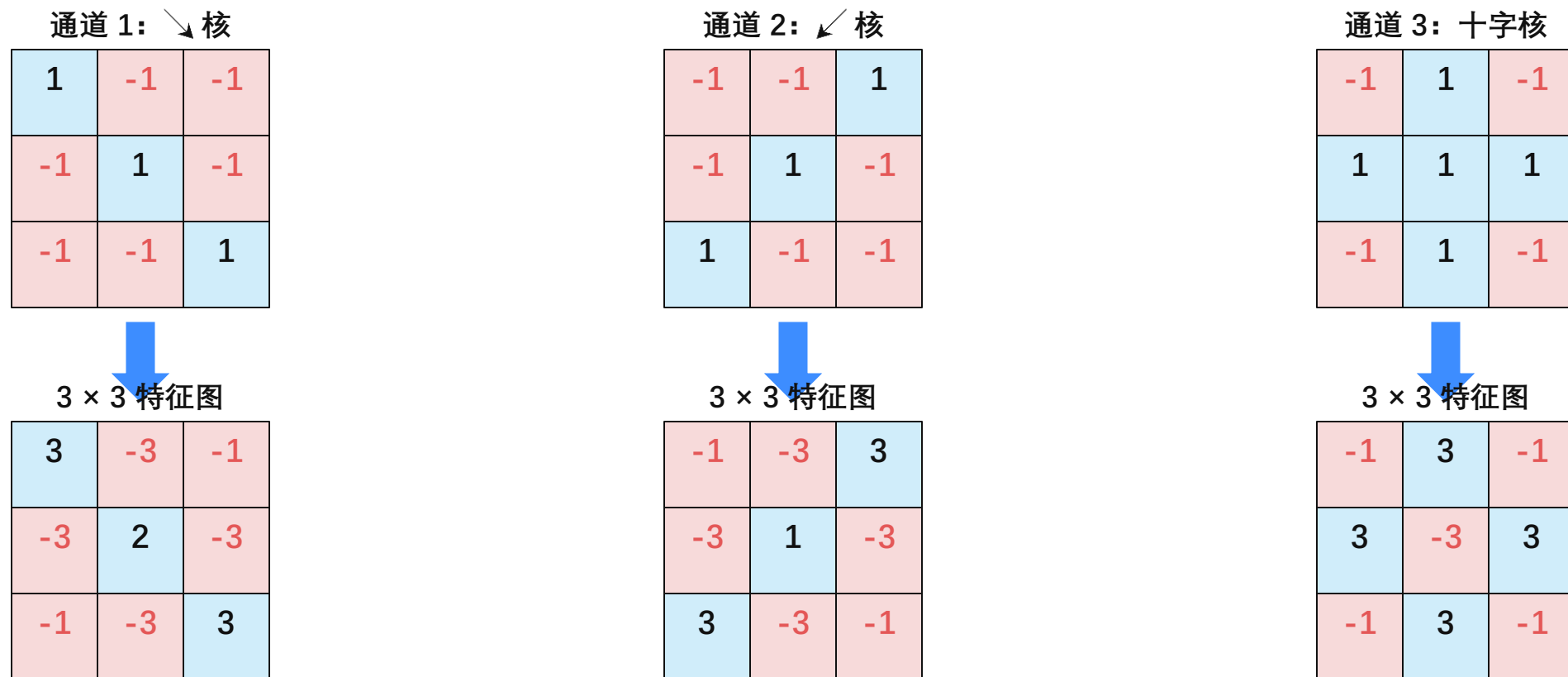
边界位置也产生响应

因为卷积核可以“部分落在图外”，图外部分按 0 参与计算。

$$N_{out} = \text{floor}((N - K + 2P) / S) + 1$$

$$N=5, K=3, P=1, S=1 \rightarrow N_{out}=5$$

三个卷积核产生三个特征通道



1 个卷积核 → 1 个输出通道 | 3 个卷积核 → 3 个输出通道

堆叠后的张量尺寸: $3 \times 3 \times 3$

池化

`nn.MaxPool2d(kernel size, stride, padding)`

例: `nn.MaxPool2d(2, 2, 1)`: 窗口大小 2×2 , 步长=2, 填充=1

step1

3	0	0
0	2	0
0	0	3

最大值

2×2 池化结果

3	

每块只留下 “最强证据 ”

step2

0	0	0	0	0
0	3	0	0	0
0	0	2	0	0
0	0	0	3	0
0	0	0	0	0

最大值

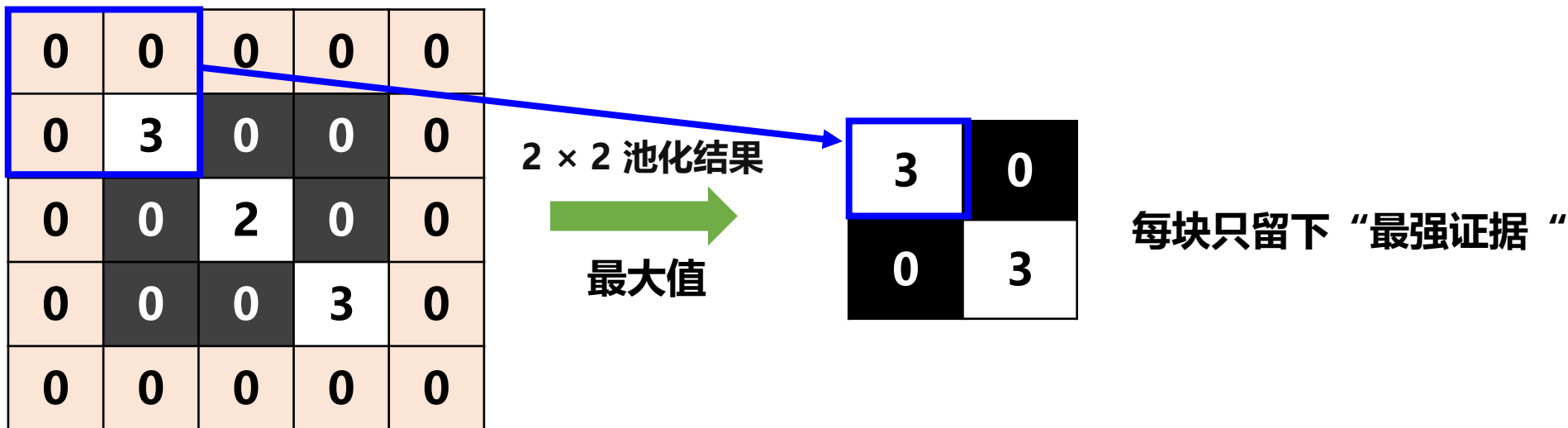
3	0
0	3

padding=1

池化

`nn.MaxPool2d(kernel size, stride, padding)`

例: `nn.MaxPool2d(2, 2)`: 窗口大小 2×2 , 步长=2, 填充=1



分两步理解:

1. 先 padding: 3×3 四周各补一圈 0 $\rightarrow$ 变成 5×5

2. 再池化: 5×5 用 2×2 、stride=2 $\rightarrow \lfloor (5-2)/2 \rfloor + 1 = 2$, 得 2×2

注意: 这里结果是 2.5 向下取整 (PyTorch 默认 `ceil_mode=False`), 因为 padding 后 5×5 的最右、最下一行/列 (索引 4) 放不下完整的 2×2 窗口, 被丢弃了。若设 `ceil_mode=True` 则会向上取整得到 3×3 。

卷积运算小节

输入层: 5x5 (1通道)

1	0	0	0	1
0	1	0	1	0
0	0	1	0	0
0	1	0	1	0
1	0	0	0	1

卷积+ReLU

3种卷积核
padding=0

3	0	0
0	2	0
0	0	3

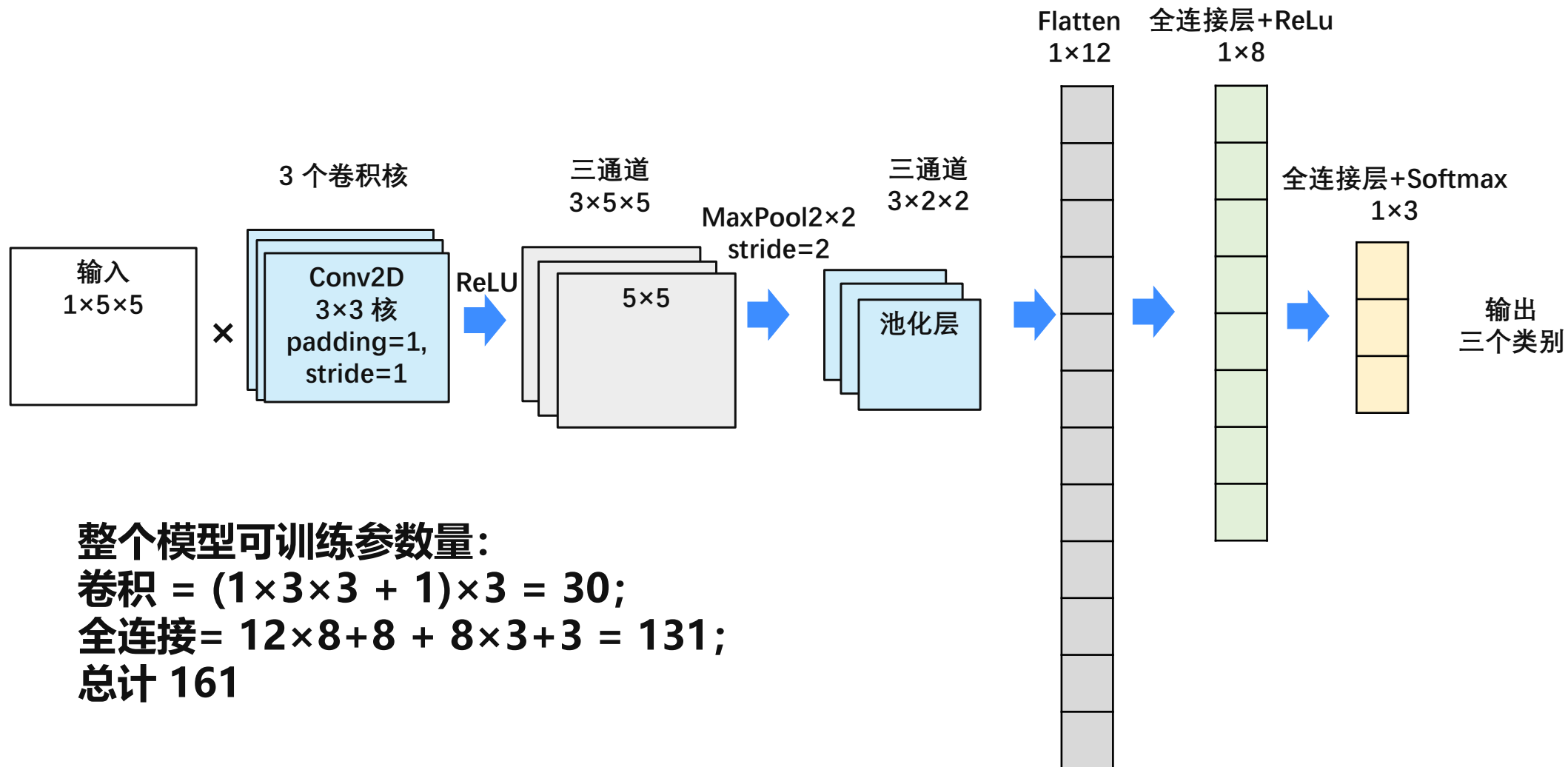
卷积层: 3x3 (3通道)

(2x2max 池化)

3	0
0	3

2x2 池化层 (3通道)

一个完整的"玩具" CNN：从 5×5 到 3 类概率



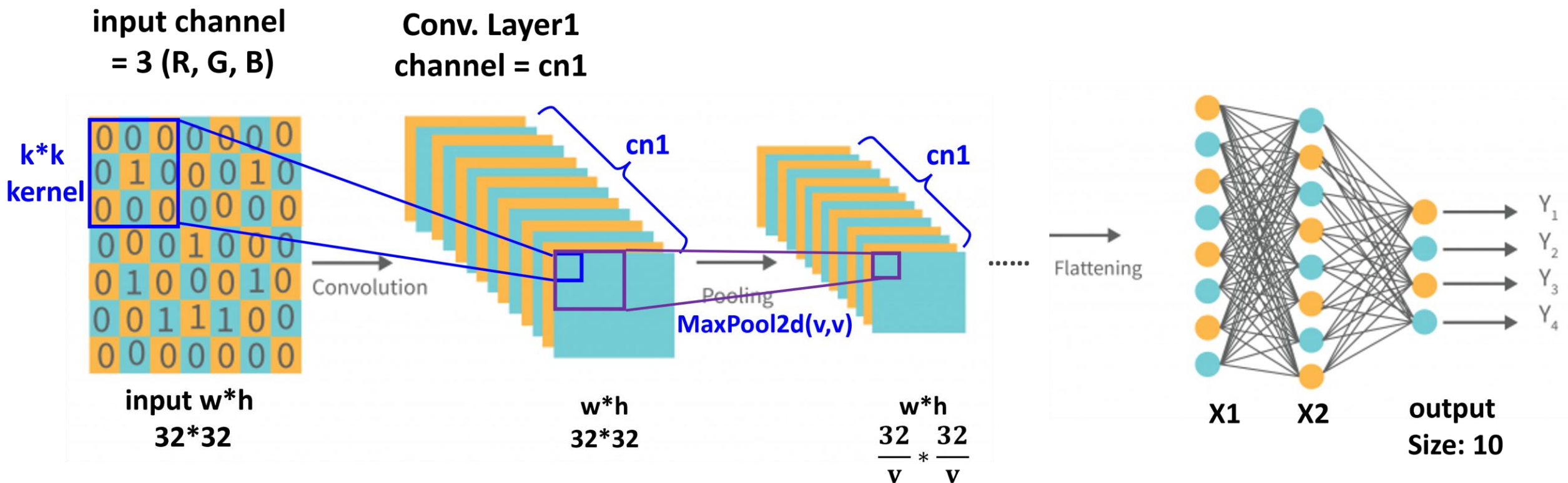
整个模型可训练参数量:

卷积 = $(1 \times 3 \times 3 + 1) \times 3 = 30$;

全连接 = $12 \times 8 + 8 + 8 \times 3 + 3 = 131$;

总计 161

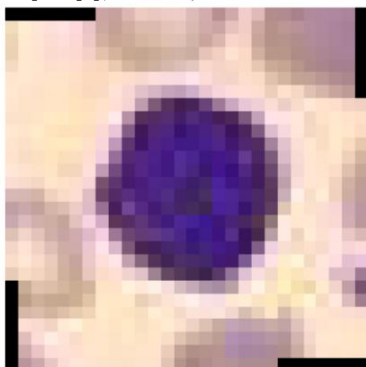
彩色 $3 \times 32 \times 32$ 图片的一个样例CNN



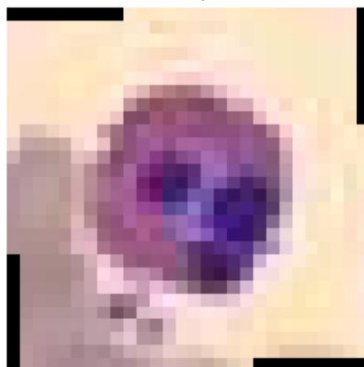
CNN分类BloodMNIST数据集

- **BloodMNIST**数据集：17,092 张彩色血液细胞纤维图片，其中训练集 11,959，验证集 1,712，测试集3,421；共8类血液细胞。
- 每个彩色图片图片： $3 \times 28 \times 28$ 矩阵 (像素 28×28 ,每个像素含RGB0~225值)。

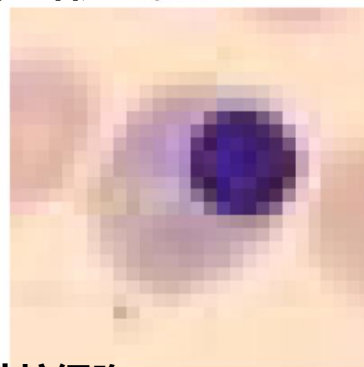
嗜碱性粒细胞 basophil



嗜酸性粒细胞 eosinophil



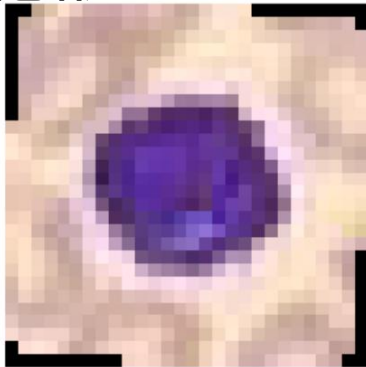
成红细胞 erythroblast



granulocytes 粒细胞



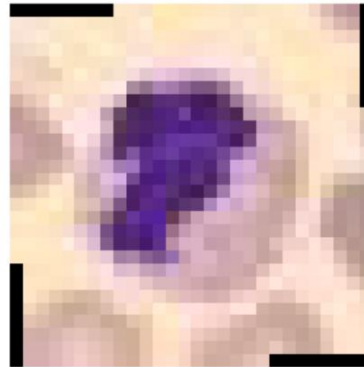
淋巴细胞 lymphocyte



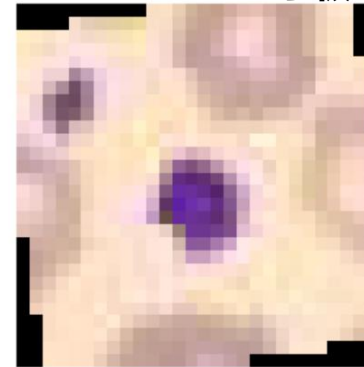
单核细胞 monocyte



中性粒细胞 neutrophil



platelet 血小板



BloodMNIST数据集加载(2.2 load data)

Data preprocessing

```
transform_train = transforms.Compose([
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(10),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5] * n_channels, std=[0.5] * n_channels),
])
```

.....

Load datasets

.....

```
train_dataset = DataClass( split="train", root=data_root,transform=transform_train,download=False)
val_dataset = DataClass(split="val",root=data_root,transform=transform_eval,download=False)
test_dataset = DataClass( split="test", root=data_root,transform=transform_eval, download=False)
```

.....

数据集查看 – 总类别 8 类

```
print("BloodMNIST classes:")  
for index, name in enumerate(class_names):  
    print(f" {index}: {name}")
```

```
BloodMNIST classes:  
0: basophil  
1: eosinophil  
2: erythroblast  
3: granulocytes  
4: lymphocyte  
5: monocyte  
6: neutrophil  
7: platelet  
One image tensor shape:  
torch.Size([3, 28, 28])  
One label shape: (1,); value: 7
```

数据集查看 - 单张图片 (单个细胞)

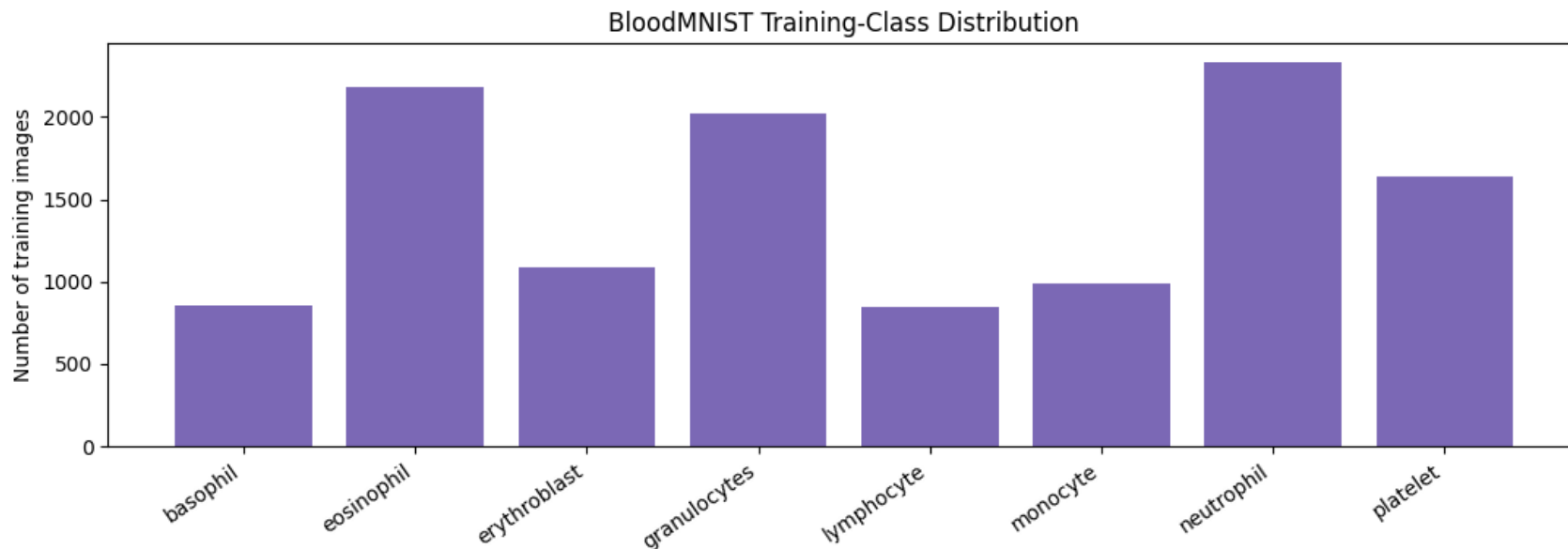
```
image_show = denormalize_blood_image(image)
plt.figure(figsize=(3, 3))
plt.imshow(image_show.permute(1, 2, 0))
plt.title(class_names[int(np.asarray(label).squeeze())])
plt.axis('off')
plt.show()
```



数据集查看 - 各类血液细胞的数量

```
# Inspect class balance in the training split
train_labels = np.asarray(train_dataset.labels).squeeze().astype(int)
class_counts = np.bincount(train_labels, minlength=n_classes)
```

```
plt.figure(figsize=(11, 4))
plt.bar(range(n_classes), class_counts, color='#7b68b5')
plt.xticks(range(n_classes), class_names, rotation=35, ha='right')
plt.ylabel('Number of training images')
plt.title('BloodMNIST Training-Class Distribution')
plt.tight_layout()
plt.show()
```



CNN代码实现--定义卷积层

```
class BloodCellCNN(nn.Module):
    def __init__(self, num_classes=8):
        super().__init__()
        self.features = nn.Sequential(
            # 卷积层1
            nn.Conv2d(3, 32, kernel_size=3, stride=1, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2), # 28 -> 14
            # 卷积层2
            nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2), # 14 -> 7
        )
```

CNN代码实现--定义卷积层

卷积层3

```
nn.Conv2d(64, 128, kernel_size=3, stride=1, padding=1),  
nn.BatchNorm2d(128),  
nn.ReLU(inplace=True),  
nn.AdaptiveAvgPool2d((3, 3)),           #7-> 3(designated)  
)
```

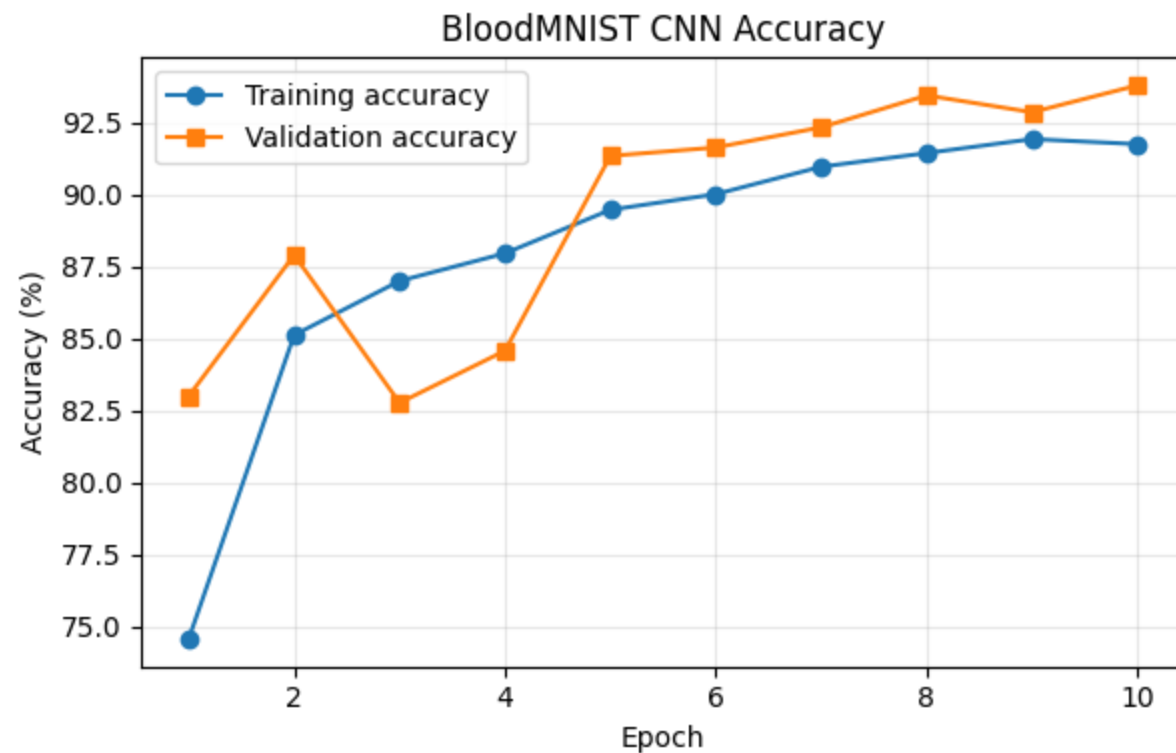
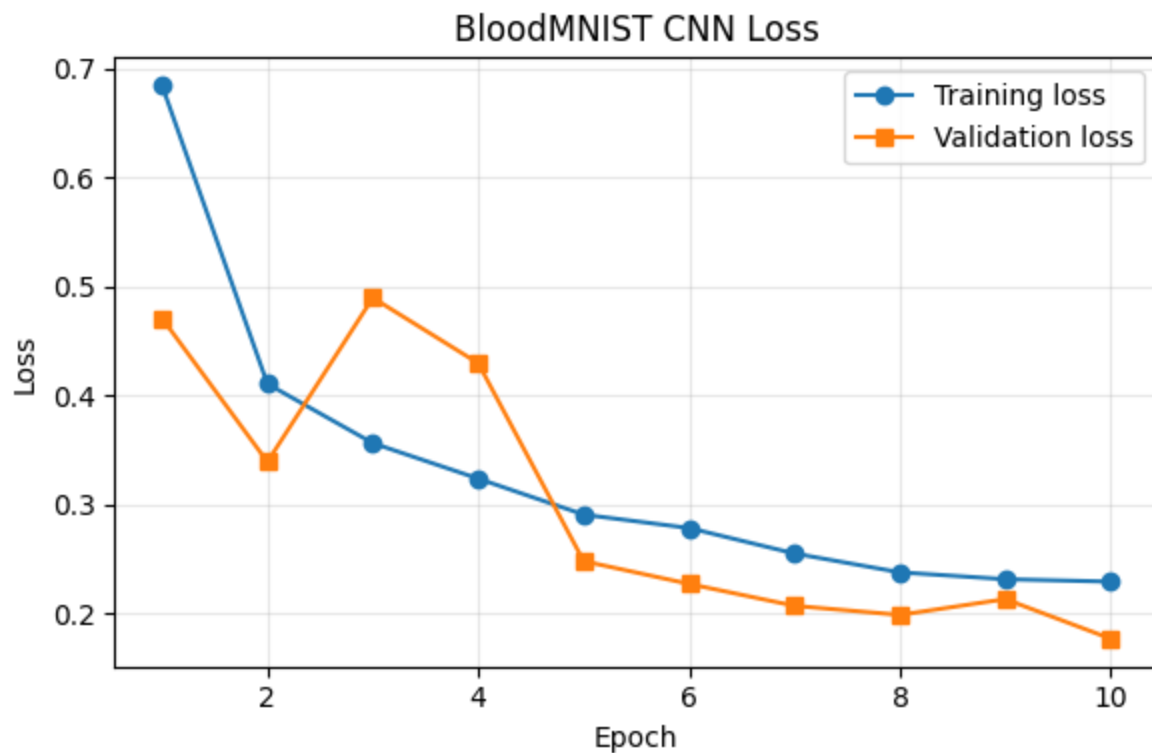
全连接层

```
self.classifier = nn.Sequential(  
    nn.Flatten(),  
    nn.Dropout(0.30),  
    nn.Linear(128 * 3 * 3, 128),  
    nn.ReLU(inplace=True),  
    nn.Dropout(0.30),  
    nn.Linear(128, num_classes),
```



输出层: 8个类别

模型训练情况



TODO

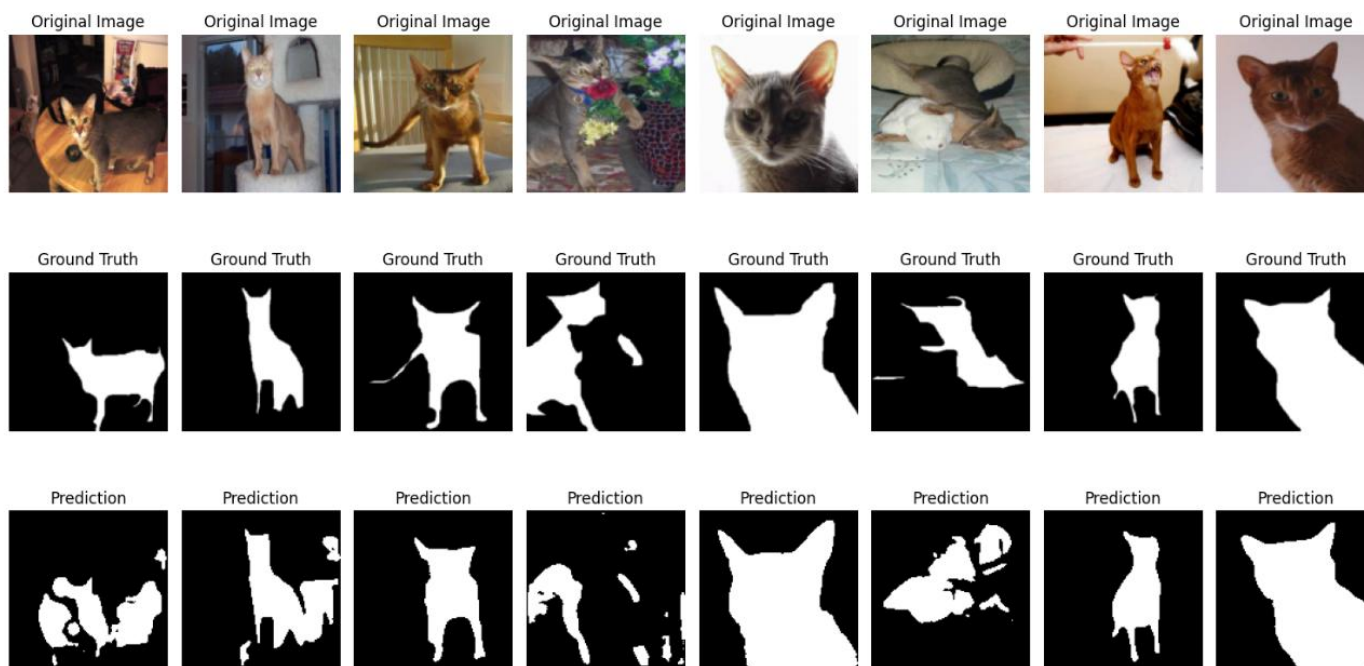
1. 将同样结构的CNN模型在GPU上训练，记录训练所需时间。
2. 调整训练算法，观察训练所需总时长，绘制loss vs epoch和accuracy vs epoch图，对比不同算法。（二选一）
 - 将损失函数由交叉熵损失`nn.CrossEntropyLoss()`换成均方误差损失`nn.MSELoss()`
 - 将非线性激活函数由relu换成sigmoid
3. 搭建一个CNN模型训练MNIST手写数字模型，并用训练好的模型预测自制手写数字图片。
4. 对比CNN模型和MLP模型的准确度和训练时长。

Lab2 Plus: 图片分割Segmentation (了解)

图像分割是计算机视觉中的一项重要技术，旨在将图像**划分为多个互不相交的区域**，以便更好地理解和分析图像内容。

- 对每一个像素点进行分类：背景或前景？获得物体精确的边界：

U-Net Pet Segmentation Results
(White=Foreground, Black=Background)

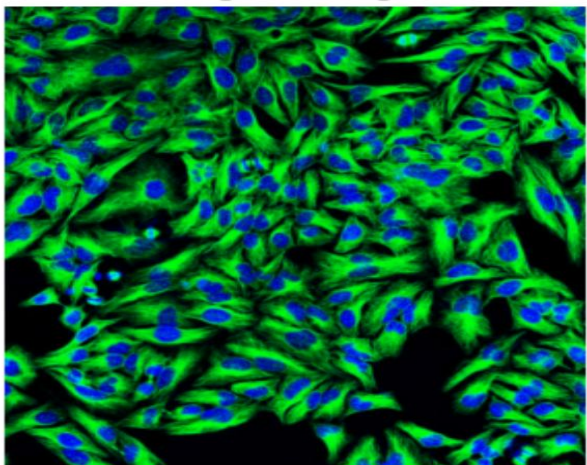


- 阈值法
- 聚类法
- 边界分割
- 直方图法
- ...

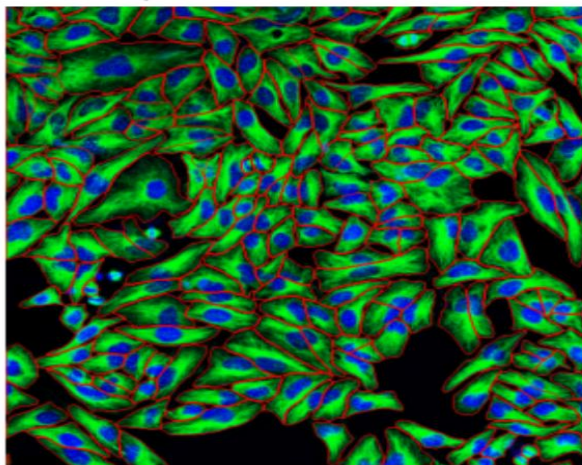
图片分割Segmentation

获得目标的精确边界：一个细胞、多个细胞？ 细胞面积？ 荧光信号？

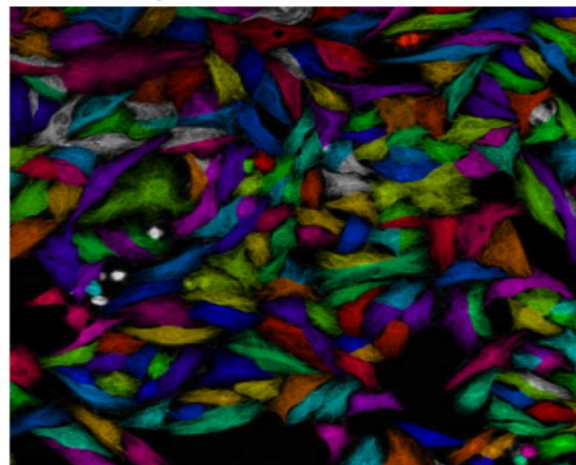
original image



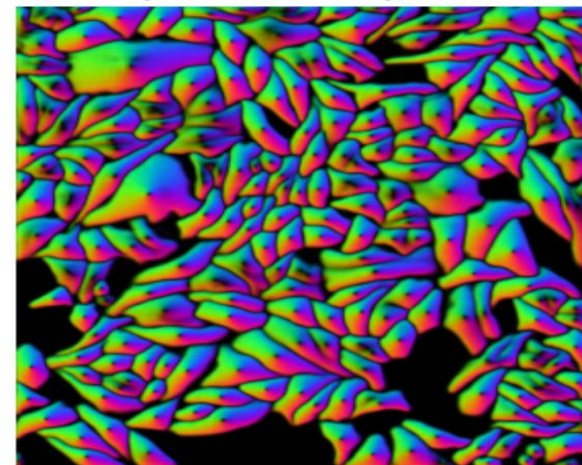
predicted outlines



predicted masks



predicted cell pose

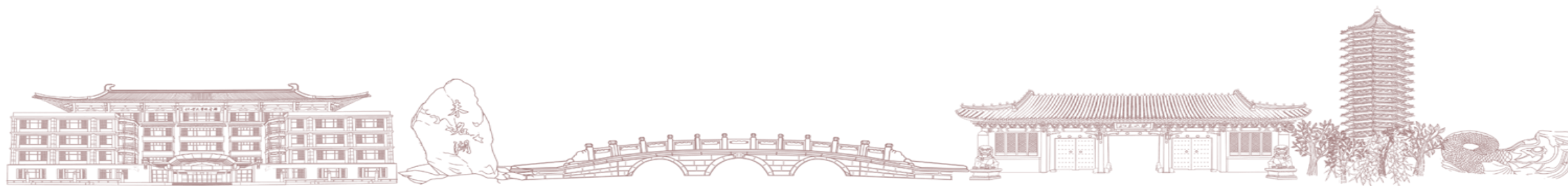




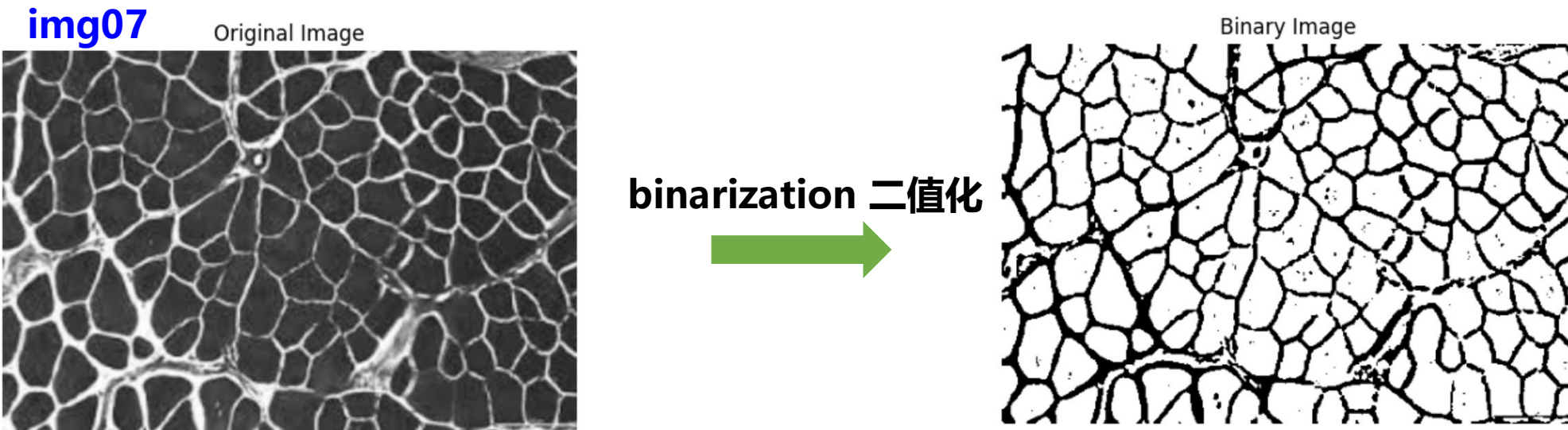
实验三、细胞分割大模型在细胞图像分析中的应用

Lab 1: Introduction to Cell Segmentation

Jupyter notebook: lab3_intro2CellSeg.ipynb



Task1: 用OpenCV进行细胞分割 – 二值化



```
blocksize = 35  
C = -5  
# Apply adaptive thresholding  
image_binary = cv2.adaptiveThreshold(image, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,  
cv2.THRESH_BINARY_I
```

目的：区分前景（白色）和背景（黑色）（细胞分割常用设置）

注意：对于彩色细胞图片 需用 THRESH_BINARY

用OpenCV进行细胞分割 – 二值化

- 二值化目的：区分前景和背景
- `cv2.adaptiveThreshold` () 参数：
 - **blocksize**:是计算局部阈值的邻域窗口大小（奇数，如3, 5, 11等）。算法会在图像的每个局部区域内独立计算阈值。
 - **C**: 从局部均值中减去的常数（可正可负），用于微调阈值：阈值 = 均值 - C

参数	增大值的影响	减小值的影响
Blocksize	- 阈值更鲁棒，适应光照不均 - 可能丢失细小细节（如边缘） - 计算速度稍慢	- 保留更多细节。 - 对噪声更敏感。 - 可能无法处理大范围光照变化
C	- 若C为正：阈值降低，更多像素判为前景 - 若C为负：阈值升高，前景减少（更严格）	- 若C为正：阈值升高，前景减少 - 若C为负：阈值降低，前景增多

用OpenCV进行细胞分割 – 形态学操作



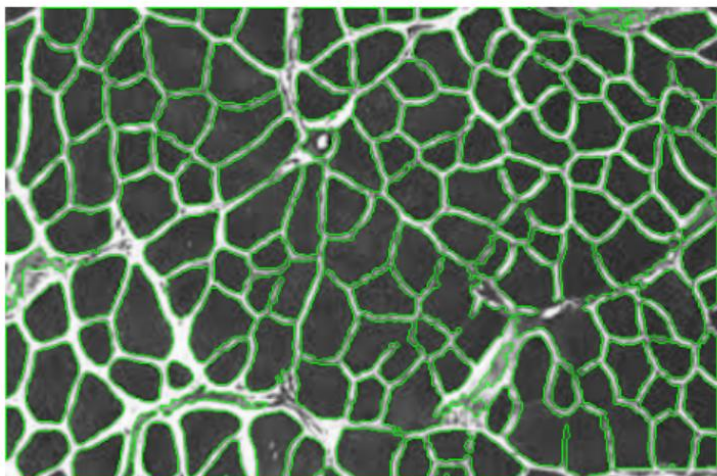
```
kernel = np.ones((2, 2), np.uint8)
# Perform morphological operations
image_opened = cv2.morphologyEx(image_binary, cv2.MORPH_OPEN, kernel)
```

去噪、补洞、区分相连的物体

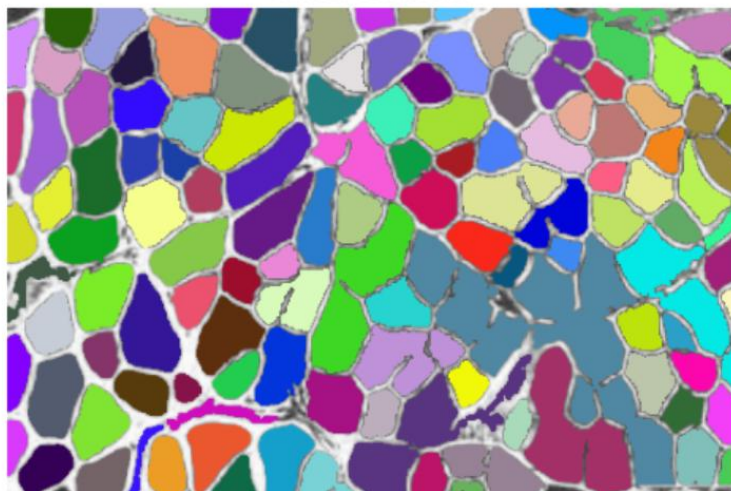
调参数结构元素窗口: `kernel = np.ones((2, 2),`

用OpenCV进行细胞分割-边缘、填充、统计

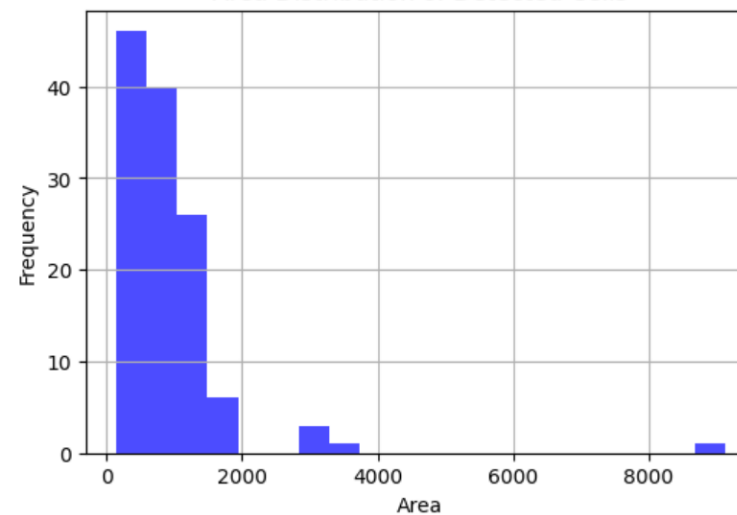
Contours



Filled Contours



Area Distribution of Detected Cells



TODO:

尝试两个其他细胞图片，调整参数到你认为最优，记录参数、细胞总数和分割质量。

1. Img22
2. 其他彩色图片

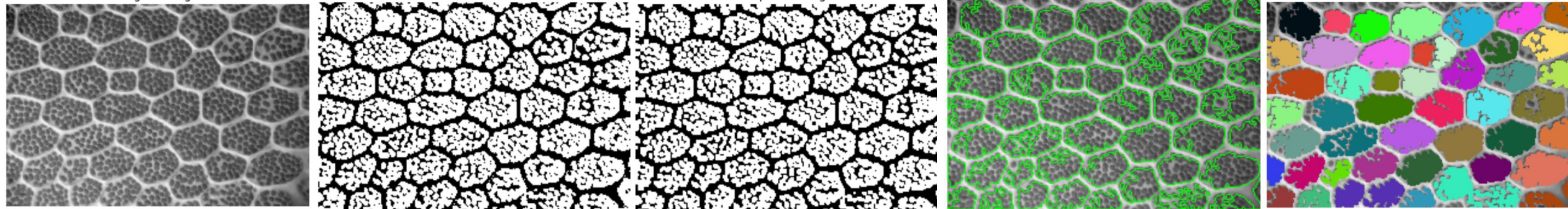
Original Image

Binary Image

Opened Image

Contours

Filled Contours



Cellpose SAM 细胞分割预训练模型



a generalist algorithm for cellular segmentation



Check out full documentation [here](#). For software advice, check out our topic on [image.sc](#). Download the Cellpose dataset [here](#).

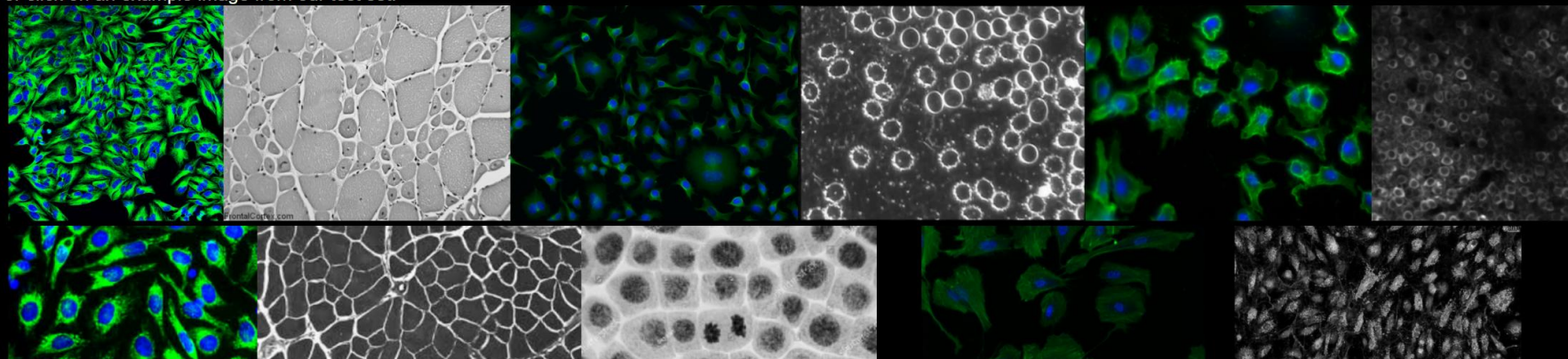
✨ NEW RELEASE: [Cellpose-SAM: superhuman generalization for cellular segmentation](#) ✨

Try out *Cellpose-SAM* on our 🤗 HuggingFace [space](#)!

You can also run Cellpose-SAM here by uploading one PNG or JPG <10 MB. Images are resized to a max size of **224x224** pixels, and runtime is ~75 seconds because it runs on only 1 CPU core.

Drop files here or click to upload.

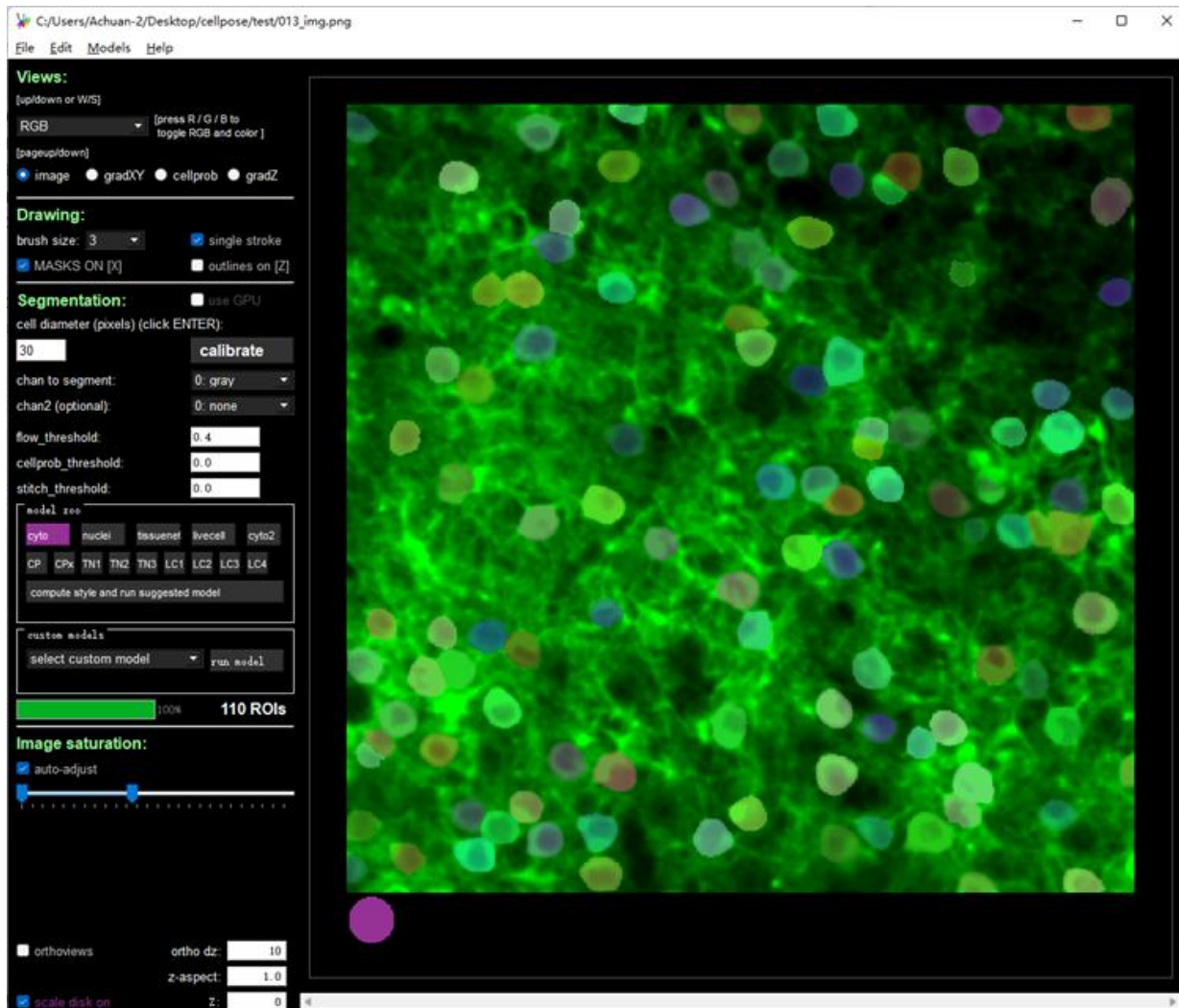
or click on an example image from our test set:



carsen stringer & marius pachitariu

<https://www.cellpose.org/>

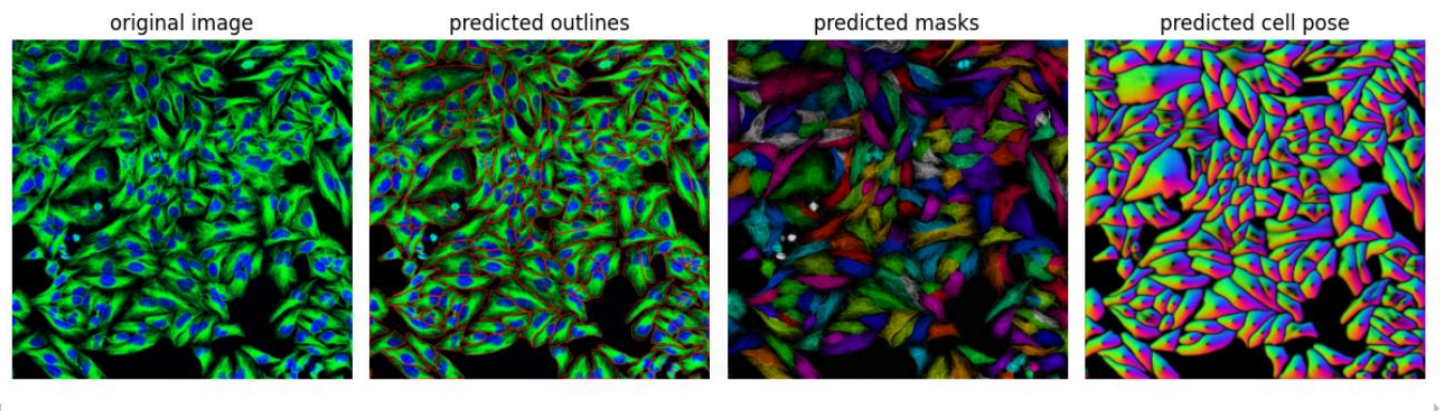
Task 2. Cellpose 图形可视化界面 (选做)



- 网页端直接使用
- 本地图形界面：需用本地机器运行；需要安装 Anaconda(或轻量版：miniconda)；(选做)

Task3: 用Cellpose预训练模型进行细胞分割

- 使用Cellpose API（接口）调用大模型
- 第一次使用需要VPN，从hugging face下载模型cpsam
- 波尔平台已部署好模型



TODO:

- Change image url = "<https://www.cellpose.org/static/images/img22.png>", and filename = "./cell_img22.png" and run task again.
- Compare the segmentation results of img22 by openCV and Cellpose. Which is more robust and accurate?

实验四：预告

- **使用Cellpose图形界面，依托MATLAB使用**
 - 较为用户友好
 - 不需要过多编程
 - 提前安装MATLAB
- **备选方案：**
 - 使用Python编程链接Cellpose大模型，通过命令行操作



北京大学
PEKING UNIVERSITY

~前路漫漫、行则将至~

欢迎交流讨论~

13366803606

HUHUUMU PHOTOGRAPHY

Huhu is naughty. Hui and Huihui is a lovely girl.
Huihui is the nickname given by Huhu.
Huhu always smokes.
So Huihui called him Huhu to return.
They will keep mutual care, cherish, like this and purely stand down.